

Constraint-First Reasoning: A Training-Free Protocol for Exploiting Answer-Space Constraints in Mathematical Problem Solving

Hongbo Ma^{1,*}, Bangji Yang^{2,*}, Yunqian Selina Cheng^{1,*},
Jiajun Fan², Hanwen Zhang¹, Ge Liu^{2,†}

¹Tsinghua University ²University of Illinois Urbana-Champaign

*Equal contribution †Corresponding author

Abstract

Large language models can derive a plausible mathematical object yet still violate explicit requirements—for example, by omitting a modular reduction, returning a non-integer, or using the wrong encoded answer form. We introduce **Constraint-First Reasoning (CFR)**, a training-free two-stage prompting protocol: Stage 1 extracts and summarizes constraints entailed by the problem, and Stage 2 solves while checking intermediate and final results against that summary. ROUTED-CFR activates the two-stage protocol only when a text-only regex router detects restrictive cues; otherwise it uses direct chain-of-thought (CoT). Across AIME, CMIMC, BRUMO, and AIMO_AMC, the method improves direct CoT on multiple backbones. We further report convention-controlled routing experiments, matched prompting baselines, problem-level paired tests, decoding robustness, constraint-quality audits, total-token accounting, and an OlympiadBench evaluation. These analyses position CFR as a targeted test-time intervention whose benefit depends on recoverable constraints and reliable Stage 1 extraction, rather than as a general-purpose replacement for mathematical reasoning.

1 Introduction

Competition mathematics problems embed constraints in natural language: “find the remainder when N is divided by 1000,” “how many positive integers satisfy...,” “express your answer as $p + q$ where p and q are coprime.” These constraints define the *feasible answer region*—a small, structured subset of all possible values. A solver that ignores them risks producing answers that violate stated requirements despite correct underlying reasoning.

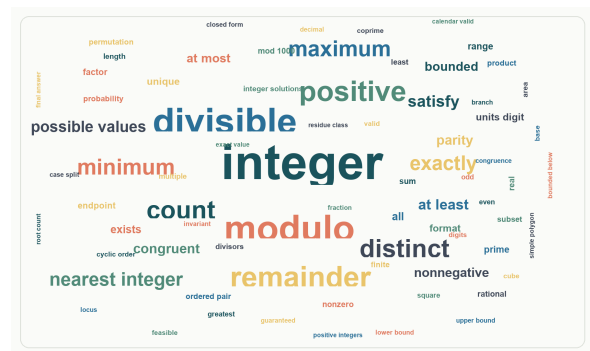


Figure 1: **Restrictive lexical cues in competition mathematics.** Larger words correspond to cue types emphasized by our router, including integer-answer formats, divisibility and modular conditions, bounds, extremal operators, exactness requirements, and counting restrictions. These cues motivate CFR: before solving, identify what any valid answer must satisfy.

Why do constraint violations occur so frequently? We identify three root causes. First, *attention dilution*: in long reasoning chains, early constraint information can fade before the model reaches a final answer (Liu et al., 2024). Second, *implicit encoding*: models trained on solution traces handle constraints implicitly—they may notice a modular condition mid-solution or check integer constraints post-hoc—but this implicit handling breaks under distribution shift. Third, *format ambiguity*: problems with encoded answer formats (“find $m + n$ where $\gcd(m, n) = 1$ ”) require multi-step post-processing that models frequently skip or misapply.

Current reasoning models, trained to generate extended chains of thought (DeepSeek-AI, 2025; OpenAI, 2024), handle constraints implicitly. A model may follow a plausible derivation while leav-

ing answer restrictions implicit until the end; explicit constraint tracking is designed to keep those restrictions visible throughout solving.

We propose **Constraint-First Reasoning (CFR)**, a training-free protocol that makes constraint handling explicit and systematic. The key insight: *extract constraints before solving, not after*. This mirrors human competition practice, where experienced contestants identify the required answer form before attempting a solution (Pólya, 1945). Front-loading constraint identification ensures the solver maintains awareness of answer-space requirements throughout its reasoning chain.

CFR operates in two stages:

1. **Constraint Extraction & Prompted Summary:** Extract answer-space constraints (domain, modular, parity, bounds, format) and problem-structure constraints (e.g., invariants or branch conditions) from the problem statement, then summarize their implications for solving.
2. **Constraint-Guided Solving:** Solve under the extracted constraints, checking intermediate results against the constraint specification at each major step.

We further introduce **ROUTED-CFR**, which prepends a regex-based router that checks whether the problem text contains restrictive lexical cues. Problems without such cues bypass the pipeline and proceed directly to standard chain-of-thought reasoning. This purely syntactic router decides whether to use the full pipeline before generation begins.

Our experiments use Table 1 and Table 3 as the primary empirical evidence. Detailed statistical, router, and error analyses are reported in the appendix. They reveal a clear and interpretable pattern:

- Under the text-only router, Table 1 shows positive AIME/CMIMC average gains for all four evaluated backbones; the magnitude varies with Stage 1 reliability.
- Table 3 compares CFR with matched planning, verification, format-only, and token-matched prompting alternatives under a single evaluation protocol.
- The appendix reports pooled problem-level analyses after removing the AIME routing convention and correcting for multiple comparisons.
- Table 3 makes the accuracy–cost trade-off explicit: routing reduces, but does not eliminate, the inference overhead of the two-stage protocol.

1.1 Evaluation Protocol and Method Labels

Unless explicitly stated otherwise, every accuracy in the main paper is **pass@1 averaged over four independent runs** (avg@4). Each run produces one answer per problem; we use the same decoding setting, answer extractor, and grading script for all methods in a comparison. We do not use self-consistency, majority voting, best-of- k selection, or answer-format filtering in these primary results. For paired analyses, the four pass@1 outcomes are first averaged within each problem, so the problem—not an individual generation—is the unit of inference. Appendix H retains a clearly labeled pass@4 exploratory comparison, which is not pooled with or used to substantiate the main pass@1 claims.

We use **Routed-CFR** for the text-only router followed by the two-stage pipeline, and **CFR (always-on)** for the same two-stage pipeline without routing. This distinction is used consistently in tables and captions. “CFR” without a qualifier refers only to the general constraint-first protocol, not to an ambiguous experimental variant.

These findings position CFR not as a universal reasoning enhancer but as a targeted intervention for settings where answer-space constraints are informative and the backbone can use them.

2 Related Work

2.1 Self-Refinement, Planning, and Verification

Self-Refine (Madaan et al., 2023) iteratively improves outputs via self-feedback. Process reward models (Lightman et al., 2024; Wang et al., 2024) score individual reasoning steps to guide search. Outcome reward models (Cobbe et al., 2021) provide binary signals on final answers. ReAct (Yao et al., 2023b) interleaves reasoning with actions for grounded decision-making, while Plan-and-Solve (Wang et al., 2023a) produces an explicit plan before solution generation. Learning-based alternatives further examine robust preference optimization, evolving sample interactions for multi-domain fine-tuning, noisy or verifier-independent reasoning rewards, and policy optimization through interactive guidance (Liang et al., 2025a,b; Cai et al., 2025; Cai and Sugiyama, 2026; Liu et al., 2026). These methods respectively emphasize planning, post-hoc refinement, verification, or policy learning. CFR is narrower: it constructs a problem-specific constraint summary before solving and

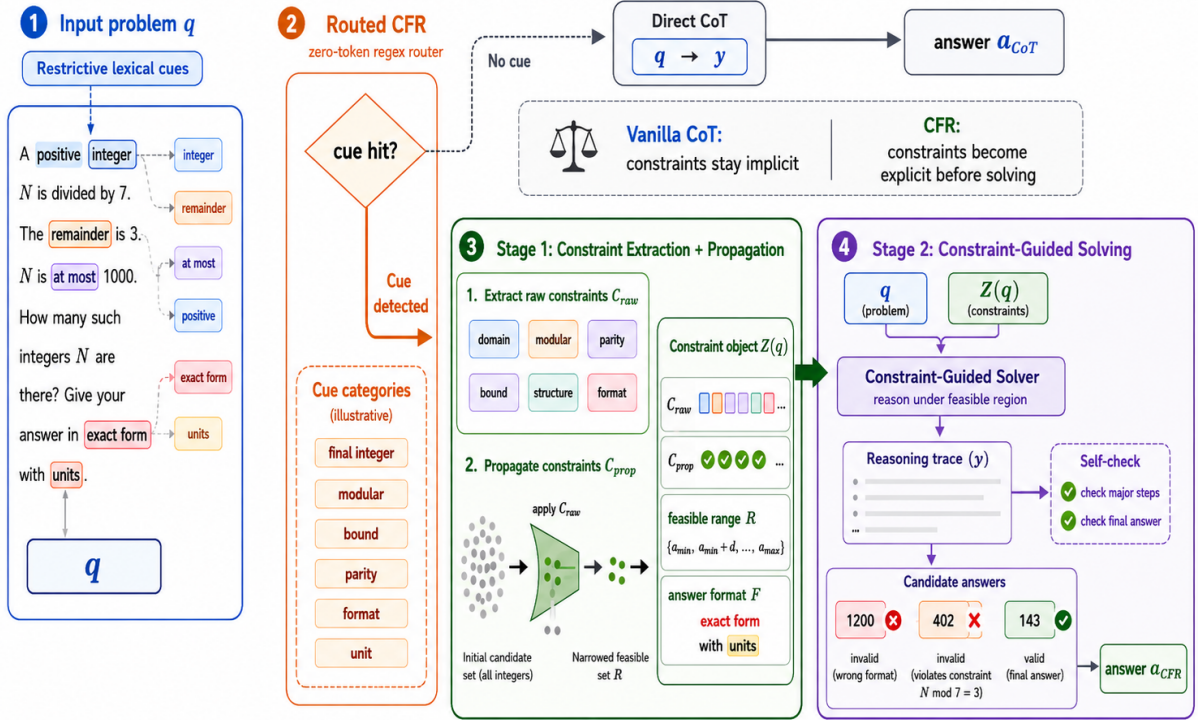


Figure 2: **ROUTED-CFR framework overview.** Given an input problem, the regex router checks for restrictive lexical cues. If detected, Stage 1 extracts and cross-propagates answer-space constraints (domain, modular, parity, bound, format), then Stage 2 solves under the resulting feasible set with interleaved self-checks. Problems without cues bypass the pipeline via direct CoT.

carries that summary into the solving prompt. It therefore does not claim to replace planning, verification, or training-based optimization pipelines; Table 3 empirically separates this design from their prompting counterparts.

2.2 Test-Time Compute Scaling

Recent work demonstrates that allocating more inference-time computation improves performance across diverse tasks. Best-of- n sampling (Brown et al., 2025) and compute-optimal scaling (Snell et al., 2025) establish scaling laws for test-time compute. S1 (Muennighoff et al., 2025) shows that budget forcing (extending reasoning with “wait” tokens) improves accuracy. Unlike generic compute scaling, CFR concentrates additional computation on constraint identification—a narrow task with compact output that produces high-value information guiding the larger solving budget.

3 Method

3.1 Problem Formulation

Let q denote a mathematical problem, $\mathcal{A}$ the answer space, y a reasoning trace produced by the model, and $\pi(y)$ the function that extracts a final answer from y . Standard inference samples:

$$a^* = \pi(y^*), \quad y^* = \arg \max_y P_\theta(y \mid q) \quad (1)$$

where P_θ is the language model. Many competition problems define a feasible answer region $\Phi(q) \subset \mathcal{A}$ through explicit constraints (e.g., “integer in 000–999” implies $|\Phi(q)| = 1000$ vs. $|\mathcal{A}| = \infty$). CFR makes $\Phi(q)$ explicit:

$$a_{CFR}^* = \arg \max_{a \in \Phi(q)} P_\theta(a \mid q, \Phi(q)) \quad (2)$$

The feasible-set notation is a conceptual description of the goal rather than a claim of formal constrained decoding. Because the Stage 1 summary is generated from q , it contributes no external evidence; its potential value is to reorganize problem

information into an explicit prompt context that can change the model’s subsequent reasoning behavior. We therefore treat the information-theoretic discussion in Appendix L as intuition, not as a derived performance guarantee.

3.2 Stage 1: Constraint Extraction and Prompted Summary

Given problem q , Stage 1 prompts the model to extract structured constraints without solving, following a fixed taxonomy:

We distinguish *direct answer-space constraints*, which directly restrict the final form, range, modularity, or option set, from *problem-structure constraints*, such as dimensional consistency, geometric invariants, and branch conditions. The latter do not necessarily specify the final answer form, but can rule out invalid reasoning paths and thereby indirectly narrow admissible answers.

- **Domain:** integrality, positivity, ranges
- **Modular:** remainder conditions, divisibility, congruences
- **Parity:** even/odd requirements
- **Bound:** inequalities, extremal values
- **Monotonic:** increasing/decreasing relationships
- **Structure:** symmetry, counting targets, geometric invariants
- **Dimension:** unit consistency, coordinate ranges
- **Format:** encoded answers ($p + q$, remainder, coprime form)

After extraction, the model derives a prompted constraint summary. For example, if C_1 : “ n is divisible by 3” and C_2 : “ $100 \leq n \leq 200$,” the summary may state that valid candidates are multiples of three between 102 and 198. The output takes the form of a structured JSON specification containing raw constraints, derived implications, the narrowest feasible range, and the required answer format.

Terminology. “Propagation” here refers only to prompt-driven natural-language summarization and checking by the same frozen LLM, not to symbolic constraint propagation or a formal constraint-satisfaction procedure. We claim neither soundness nor completeness, and evaluate the resulting reliability directly in Section 6.4.

Token budget. Stage 1 is allocated a concise budget so that constraint extraction remains compact relative to the final solving stage.

Algorithm 1 ROUTED-CFR Inference Protocol

Require: Problem q , model P_θ , budget B

```

1: cues  $\leftarrow$  REGEXROUTER( $q$ )
2: if |cues| > 0 then
3:    $C \leftarrow P_\theta(\text{extract\_constraints} \mid q)$  ▷ Stage 1
4:    $a \leftarrow P_\theta(\text{solve} \mid q, C)$  ▷ Stage 2
5: else
6:    $a \leftarrow P_\theta(\text{solve} \mid q)$  ▷ Direct CoT
7: end if
8: return  $a$ 

```

3.3 Stage 2: Constraint-Guided Solving

Stage 2 receives the original problem q and the constraint specification from Stage 1. The solver must:

1. Reason step by step toward a solution.
2. After each major step, check consistency against the constraints.
3. If an intermediate result violates a constraint, stop and re-examine.
4. Before stating the final answer, confirm all constraints are satisfied.

This contrasts with self-correction approaches ($q \rightarrow y \rightarrow \text{critique}(y)$), which verify *after* generation. CFR instead front-loads constraint awareness ($q \rightarrow C(q) \rightarrow y$), providing multiple opportunities to catch violations *during* reasoning.

3.4 Routed CFR

Not all problems benefit from constraint extraction. We introduce a regex-based router that activates CFR only when lexical cues indicate exploitable constraints.

The default router uses only the problem text; benchmark identity and benchmark-level answer conventions are not passed to either Stage 1 or Stage 2. The router consumes zero model tokens—it is a pure regex pass over the problem text. We separately audit the effect of the historical AIME convention override in Section 6.1; that ablation clarifies a routing prior without supplying solver-side information.

4 Experimental Setup

4.1 Models

We evaluate four models spanning a wide capability range:

- **DeepSeek-V4-Pro** (DeepSeek-AI, 2026): Frontier-class reasoning model.
- **Qwen3.5-35B-A3B** (Qwen Team, 2026): Strong MoE model.

- **Qwen3.5-9B** (Qwen Team, 2026): Mid-size dense model.
- **JustRL-1.5B** (He et al., 2025): Small RL-trained reasoning model.

4.2 Evaluation Scope

The empirical section uses complementary evidence rather than a single headline table. Table 1 is the convention-controlled multi-backbone comparison, Table 3 combines matched prompting baselines with total-token cost, and Table 2 is the routing ablation. Detailed paired statistical analyses, benchmark descriptions, router audits, and error analyses are provided in the appendix. Table 15 extends BRUMO25 and AIMO_AMC to all reported backbones.

Methods compared. Direct CoT is the primary baseline. Strong prompting alternatives are compared under the same protocol in Table 3. Routed-CFR uses text-only routing, while CFR (always-on) is used only for the routing ablation.

5 Results

5.1 Main Results

We begin with the convention-controlled multi-backbone comparison, then narrow the discussion to prompting controls and cost. The appendix describes the benchmark characteristics: AIME has a tight integer answer convention, while BRUMO, AIMO_AMC, and CMIMC retain explicit local constraints but vary in format and difficulty.

Table 1 provides the central result. With the AIME convention removed from routing, Routed-CFR improves the AIME/CMIMC average for each evaluated backbone. The gains are smallest for the low-capacity model and largest for DeepSeek-V4-Pro and Qwen3.5-9B, consistent with the reliability analysis in Section 6.4. Figure 3 visualizes the model–benchmark interaction, and Table 15 extends the comparison to BRUMO25 and AIMO_AMC.

Taken together, these performance tables support a conditional claim rather than a universal one: CFR can help when the problem contains recoverable constraints and the backbone can use the Stage 1 summary. Points near ceiling accuracy naturally have little room to move, while lower-capacity models can be harmed by incomplete or invalid extraction.

5.2 Matched Prompting Baselines

Table 3 compares CFR with stronger training-free prompting alternatives on DeepSeek-V4-Pro under the same pass@1 avg@4 protocol. These controls separate constraint-first guidance from a final-answer reminder, simply listing constraints, longer free-form reasoning, planning, format-only extraction, and post-hoc refinement. Text-only Routed-CFR is best on AIME24 and AIME25, ties the best AIME26 score, and remains competitive on the near-ceiling AIMO_AMC setting.

5.3 Convention Control and Paired Statistics

To address the small size of the annual AIME sets, we conduct problem-level paired analyses after disabling the AIME-specific routing convention. For every problem, we average four independent pass@1 outcomes, compute the within-problem difference, construct paired bootstrap confidence intervals, and apply a two-sided paired randomization test. The full DeepSeek-V4-Pro and Qwen3.5-9B results appear in Appendix A.1. The pooled gains remain significant after Holm correction for both models; individual annual sets have wider intervals and are therefore interpreted as supporting evidence rather than independent confirmation of a universal effect.

5.4 Ablation Results

Table 2 isolates the routing decision for DeepSeek-V4-Pro. The token multipliers explain why routing is not merely a convenience: Routed-CFR uses lower relative token cost than the always-on variant on every reported benchmark ($1.58\times$ vs. $1.71\times$ on BRUMO25, $2.08\times$ vs. $2.45\times$ on AIMO_AMC, and $1.64\times$ vs. $1.94\times$ on CMIMC25). Thus, routing can improve the accuracy–cost trade-off by applying the full protocol only when the problem text suggests useful constraints.

5.5 Interpretation

The remaining figures and tables explain the pattern behind the headline numbers. Figure 3 separates gains by model, while the appendix provides benchmark characteristics and a detailed applicability plot. Together with the constraint audit in Section 6.4, the evidence supports a limited interpretation: constraints are helpful when they are explicit enough to recover, restrictive enough to reject plausible wrong answers, and paired with a backbone capable of using them.

Model	AIME24	AIME25	AIME26	CMIMC25	Avg.
Baselines: Math LLMs					
Qwen3-1.7B	40.2%	46.7%	38.1%	20.0%	36.3%
Qwen3-4B-Thinking-2507	83.3%	66.5%	70.3%	57.5%	69.4%
Qwen3-8B	73.6%	71.5%	69.2%	52.2%	66.6%
Baselines and Ours: JustRL-1.5B					
JustRL-1.5B	47.5%	37.5%	40.0%	17.5%	35.6%
Routed-CFR-JustRL-1.5B (Ours)	47.5%	45.8%	42.5%	18.1%	38.5%
Δ vs. JustRL-1.5B	0.0%	+8.3%	+2.5%	+0.6%	+2.9%
Baselines and Ours: Qwen3.5-35B-A3B					
Qwen3.5-35B-A3B	87.5%	83.3%	87.5%	30.6%	72.2%
Routed-CFR-Qwen3.5-35B-A3B (Ours)	90.0%	84.2%	92.3%	34.3%	75.2%
Δ vs. Qwen3.5-35B-A3B	+2.5%	+0.9%	+4.8%	+3.7%	+3.0%
Baselines and Ours: Qwen3.5-9B					
Qwen3.5-9B	77.5%	61.7%	65.0%	20.0%	56.1%
Routed-CFR-Qwen3.5-9B (Ours)	81.7%	70.8%	78.3%	22.5%	63.3%
Δ vs. Qwen3.5-9B	+4.2%	+9.1%	+13.3%	+2.5%	+7.2%
Baselines and Ours: DeepSeek-V4-Pro					
DeepSeek-V4-Pro	90.8%	80.8%	83.3%	28.1%	70.8%
Routed-CFR-DeepSeek-V4-Pro (Ours)	95.8%	91.7%	92.5%	36.9%	79.2%
Δ vs. DeepSeek-V4-Pro	+5.0%	+10.9%	+9.2%	+8.8%	+8.5%

Table 1: Convention-controlled performance comparison across AIME and CMIMC. Routed-CFR uses the text-only router; the AIME benchmark convention is disabled. All values are pass@1 averaged over four independent runs. Δ denotes the gain over the matched direct-CoT backbone.

Model	BRUMO25		AIMO_AMC		CMIMC25	
	Acc.	Token	Acc.	Token	Acc.	Token
DeepSeek-V4-Pro	56.7%	1.00x	97.5%	1.00x	28.1%	1.00x
Routed-CFR-DeepSeek-V4-Pro (Ours)	60.0%	1.58x	97.5%	2.08x	36.9%	1.64x
CFR-DeepSeek-V4-Pro (always-on)	57.8%	1.71x	96.1%	2.45x	34.2%	1.94x

Table 2: Routing ablation on DeepSeek-V4-Pro. Routed-CFR uses the text-only router; CFR (always-on) runs both stages for every problem. Token values are relative to direct CoT on the same benchmark.

The bar plot in Figure 3 makes the model–benchmark interaction easier to read than the raw table. Qwen3.5-9B and DeepSeek-V4-Pro show the largest controlled AIME gains, whereas the other backbones have smaller but positive average gains. This is an empirical pattern, not a derived scaling law: Section 6.4 shows that Stage 1 quality is a plausible moderator of the observed effects.

6 Analysis

The results above show where CFR helps; the analysis below examines when this conclusion is credible. We audit the router, the scope of text-only routing beyond AIME, Stage 1 reliability, total token cost, and the error categories associated with successful interventions.

6.1 Router Behavior

The AIME answer convention can be a routing prior, but it must not be confused with solver-side information. The convention was never inserted into Stage 1, Stage 2, or direct-CoT prompts. More importantly, the controlled results in this paper disable it entirely: the text-only router still activates for 83 of 90 AIME problems. The detailed routing control and paired tests are reported in Appendix A.2.

Table 4 extends the audit to benchmarks with heterogeneous answer requirements. The non-trivial activation rates on GSM8K, MATH-500, HMMT, and OlympiadBench show that the router is not restricted to a fixed 000–999 answer convention; they do not, by themselves, establish that every routed

Method	Accuracy (%)				Mean tokens per sample			
	AIME24	AIME25	AIME26	AIMO_AMC	Input	Output	Total	vs. CFR
Direct CoT	90.8	80.8	83.3	97.5	305	13,956	14,261	0.58×
CoT + final-answer constraint reminder	87.5	88.3	89.2	96.9	—	—	—	—
List-constraints-first	91.7	89.2	89.2	98.5	—	—	—	—
Extended CoT (token-matched)	91.7	89.2	91.7	97.4	321	21,294	21,615	0.88×
Plan-and-Solve	90.0	90.8	91.7	98.0	—	—	—	—
Format-only extraction + solve	86.7	87.5	92.5	98.5	679	21,428	22,107	0.90×
Self-Refine	89.2	85.8	87.5	95.4	3,801	33,044	36,845	1.50×
CFR (always-on) [‡]	88.9	88.9	91.1	98.0	1,927	25,338	27,265	1.11×
Text-only Routed-CFR (ours)	95.8	91.7	92.5	97.5	1,736	22,336	24,072	0.98×

Table 3: Matched prompting and accuracy–cost comparison on DeepSeek-V4-Pro. Accuracy values are pass@1 averaged over four independent runs. Token columns are mean per-sample profiling measurements; “—” denotes methods for which cost profiling was not available. [‡]The always-on row is included only to quantify the routing cost trade-off.

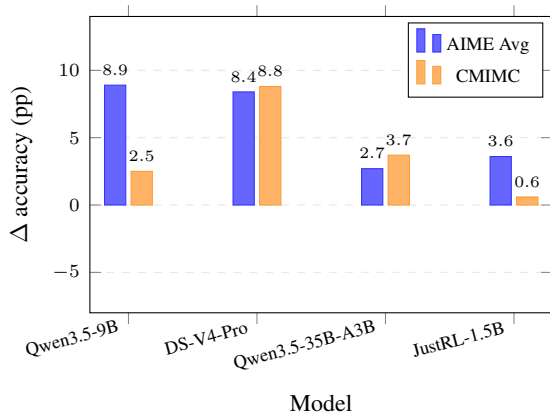


Figure 3: CFR gain (Δ accuracy) by model, computed directly from Table 1. AIME values are the mean of the three annual AIME gains; CMIMC values are the corresponding CMIMC25 gains.

Benchmark	Routed / total	Rate
GSM8K	834 / 1,319	63.23%
MATH-500	281 / 500	56.20%
AMC23	32 / 40	80.00%
HMMT 2025	21 / 30	70.00%
OlympiadBench	417 / 674	61.87%

Table 4: External audit of the text-only router.

problem benefits from CFR.

6.2 Generalization beyond Unified Answer Conventions

The AIME convention is especially compatible with answer-space constraints, so AIME alone cannot establish generalization. We therefore evaluate on OlympiadBench with all benchmark-specific conventions disabled. Its heterogeneous problem sources and answer formats provide a complementary setting in which routing depends only on prob-

lem text: Text-only Routed-CFR reaches 91.14% versus 90.66% for direct CoT, with a 61.87% routing rate. The detailed result appears in Appendix A.2; Table 15 provides further evidence on CMIMC25, BRUMO25, and AIMO_AMC. These results do not claim effectiveness on proof-oriented tasks; rather, they show that the mechanism can apply when local domain, modular, range, unit, encoding, or structural restrictions are recoverable without a shared benchmark convention.

6.3 Constraint Category Analysis

Next, we ask which extracted constraints are most responsible for the AIME gains. The appendix reports category frequency and the associated gain. Modular constraints show the largest descriptive marginal benefit (+8.7 pp), even though domain constraints occur in every AIME problem. This pattern suggests that frequency alone does not determine usefulness: modular and encoded-format constraints often require an explicit final transformation that benefits from being kept active throughout the solution.

6.4 Stage 1 Constraint Reliability

The framework is training-free and can be applied to different backbones without parameter updates, but it is not performance-agnostic: useful gains depend on Stage 1 producing a sufficiently complete and valid constraint summary. We therefore manually compare extracted constraints with those entailed by the original problem across models of different capacities. Table 7 distinguishes the number of initially extracted constraints, the number carried into Stage 2, the validity of carried constraints, and hallucination rate. The smallest model

Table 5: A representative positive case where CFR corrects CoT on DeepSeek-V4-Pro.

Example: Ratio-and-Reflection Geometry (AIME 2025)		31.1% token reduction
Problem. In $\triangle ABC$, points D, E lie on $\overline{AB}$ and F, G lie on $\overline{AC}$. Given $AD = 4$, $DE = 16$, $EB = 8$, $AF = 13$, $FG = 52$, $GC = 26$, let M be the reflection of D through F and N the reflection of G through E . If $[DEGF] = 288$, find $[AFNBCEM]$.		
Ground Truth: 588	CoT: 33,065 tokens	CFR: 22,791 tokens
DeepSeek-V4-Pro CoT	Routed-CFR-DeepSeek-V4-Pro	
CoT notices that D, F and E, G divide the two sides in the same ratios, but keeps the solution as a free-form geometric narrative. It never turns $[DEGF] = 288$ into the decisive constraint on $\sin A$, and the reflected points M, N are not explicitly encoded. The heptagon area therefore drifts away from the constrained construction. Answer: $\boxed{1} \times$	CFR makes the constraints explicit before solving: $\frac{AD}{AB} = \frac{AF}{AC} = \frac{1}{7}$, $\frac{AE}{AB} = \frac{AG}{AC} = \frac{5}{7}$; $[DEGF] = 624 \sin A = 288 \Rightarrow \sin A = \frac{6}{13}$; and $M = 2F - D$, $N = 2E - G$. Shoelace then gives $[AFNBCEM] = 1274 \sin A = 1274 \cdot \frac{6}{13} = 588$. Answer: $\boxed{588} \checkmark$	

Table 6: A representative positive case where CFR corrects CoT on Qwen3.5-9B.

Example: Cyclic-Constraint Geometry (Qwen3.5-9B, AIME 2024)		35.3% token reduction
Problem. Rectangles $ABCD$ and $EFGH$ are drawn such that D, E, C, F are collinear. Also, A, D, H, G all lie on a circle. If $BC = 16$, $AB = 107$, $FG = 17$, and $EF = 184$, find CE .		
Ground Truth: 104	CoT: 33,015 tokens	CFR: 21,370 tokens
Qwen3.5-9B CoT	Routed-CFR-Qwen3.5-9B	
CoT sets up coordinates but does not enforce the relative vertical orientation of the two rectangles. It allows the second rectangle to remain on the same side of the baseline without checking concyclicity, so the circle constraint is applied to the wrong configuration. Answer: $\boxed{64} \times$	CFR fixes the feasible orientation: $D = (0, 0)$, $C = (107, 0)$, $A = (0, 16)$, while concyclicity forces $E = (x, 0)$, $F = (x + 184, 0)$, $H = (x, -17)$, $G = (x + 184, -17)$. The circle through A, D has center on $y = 8$, and chord HG gives center x -coordinate $x + 92$, so $(x + 92)^2 + 8^2 = 92^2 + 25^2$. Thus $x = 3$ and $CE = 107 - 3 = 104$. Answer: $\boxed{104} \checkmark$	

Model	Raw	Propagated	Validity	Halluc.
DeepSeek-V4-Pro	7.74	5.12	97.5%	15.6%
Qwen3.5-9B	4.88	3.34	93.3%	13.9%
Qwen3.5-35B-A3B	5.00	3.64	91.8%	17.8%
JustRL-1.5B	2.86	2.19	73.9%	17.5%

Table 7: Manual Stage 1 audit. Raw and propagated columns are mean counts per problem; validity and hallucination are assessed against constraints entailed by the problem statement.

extracts fewer constraints and has much lower propagated validity; this is a more direct explanation of its weaker gains than a claim that CFR works uniformly across scales.

6.5 Token Overhead

The next question is cost. Output tokens alone omit the additional Stage 1 and Stage 2 prompt context, so the merged performance–cost comparison in Table 3 reports input, output, and total tokens. The two-stage protocol costs more than direct CoT, while routing reduces the overhead relative to always-on CFR.

The overhead is not uniform across backbones. DeepSeek-V4-Pro has concise baseline traces, so

the additional constraint stage is proportionally large; Qwen3.5-35B-A3B already produces longer baseline reasoning, so the same kind of constraint prefix is a smaller relative increase. The practical conclusion is deliberately limited: routing can reduce unnecessary cost, but it does not make CFR free and should be considered only where its targeted benefit justifies the added tokens.

6.6 Capability Interaction: An Empirical Interpretation

The appendix applicability plot suggests that benefits depend jointly on problem difficulty, constraint strength, and Stage 1 reliability. Near-ceiling settings leave little room for improvement, whereas a weak model may not produce a reliable enough summary to help the solver. This is an empirical interpretation of the observed pattern, not a theoretical derivation or a claim that medium-capability models necessarily benefit most. A formal account would require an explicit interaction analysis and a model of extraction error, which are beyond the present study.

6.7 Error Analysis

Finally, the appendix error analysis examines what kinds of mistakes CFR actually fixes. We manually analyze the AIME 2024–2026 cases where CFR flips Qwen3.5-9B from an incorrect to a correct outcome.

The error distribution confirms the central mechanism. Most fixes are not cases where CFR discovers a wholly new solution strategy; they are cases where the baseline reaches a plausible mathematical object but violates the required answer format, misses a modular reduction, or fails to encode the final response correctly. The smaller arithmetic category shows a secondary benefit: once the constraints are explicit, intermediate checks can catch calculations that would otherwise slip through to the final answer.

7 Case Studies

The aggregate analyses above identify the mechanism statistically; the two case studies show it operationally. Both examples involve ordinary-looking geometry problems where the baseline begins with a plausible setup but loses a decisive constraint. CFR changes the trajectory by making the hidden feasibility conditions explicit before the final derivation begins.

Table 5 shows the first failure mode: the baseline notices the proportional structure but does not convert the area and reflection conditions into binding equations. CFR instead turns the ratios, the area constraint, and the reflected-point definitions into a compact coordinate system, making the final area computation well determined. Table 6 shows a related geometric-branch failure. The baseline sets coordinates but keeps the wrong orientation feasible; CFR uses concyclicity to fix the orientation before solving. Together, the cases illustrate why the quantitative gains concentrate on constraint-heavy benchmarks: the method does not merely ask for longer reasoning, but changes which facts remain active while the model reasons.

8 Conclusion

We introduced CFR, a training-free protocol that extracts a problem-specific constraint summary before solving, and ROUTED-CFR, a text-only router that activates it selectively. Under convention-controlled routing, Routed-CFR improves the reported AIME/CMIMC averages by +2.9 to +8.5

percentage points across the four evaluated backbones. Matched prompting baselines, paired bootstrap intervals and randomization tests, an external router audit, a Stage 1 quality analysis, and complete token accounting strengthen the interpretation of these gains. The evidence does not support a general reasoning guarantee: CFR incurs added cost, is sensitive to extraction reliability, and is most appropriate when explicit answer-space or problem-structure constraints can guide the reasoning trace. Its principal effect is the prevention of answer-format, modular, and range violations rather than generic reasoning improvement.

Limitations

The empirical claims are limited to the reported numerical-answer benchmarks and the documented prompting protocols. They do not establish a universal scaling law, formal constraint-solving guarantee, or improvement on open-ended proof tasks; the latter remain particularly challenging for current language models. CFR also depends on valid and sufficiently complete Stage 1 extraction. If the summary is incomplete or incorrect, it can bias Stage 2 toward an invalid feasible region, as suggested by the lower propagated validity for JustRL-1.5B. Finally, the router audit measures activation, not calibrated probability of benefit, and the two-stage protocol increases both input and output token use. Future work should evaluate learned or calibrated routers, annotate extraction completeness at larger scale, and test proof-oriented and open-answer settings.

References

- Debangshu Banerjee, Tarun Suresh, Shubham Ugare, Sasa Misailovic, and Gagandeep Singh. 2025. CRANE: Reasoning with constrained LLM generation. In *Proceedings of the 42nd International Conference on Machine Learning*.
- Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. 2025. Large language monkeys: Scaling inference compute with repeated sampling. In *International Conference on Learning Representations*.
- Xin-Qiang Cai and Masashi Sugiyama. 2026. Vicurl: Stabilizing verifier-independent rl reasoning via confidence-guided variance reduction. *arXiv preprint arXiv:2602.12579*.
- Xin-Qiang Cai, Wei Wang, Feng Liu, Tongliang Liu, Gang Niu, and Masashi Sugiyama. 2025. Reinforcement learning with verifiable yet noisy rewards under imperfect verifiers. *arXiv preprint arXiv:2510.00915*.
- Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W Cohen. 2023. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research*.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, and 1 others. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.
- Rina Dechter. 2003. *Constraint Processing*. Morgan Kaufmann.
- DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*.
- DeepSeek-AI. 2026. Deepseek-v4: Towards highly efficient million-token context intelligence.
- Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. PAL: Program-aided language models. In *Proceedings of the 40th International Conference on Machine Learning*.
- Ruohao Guo, Afshin Oroojlooy, Roshan Sridhar, Miguel Ballesteros, Alan Ritter, and Dan Roth. 2026a. Tree-based dialogue reinforced policy optimization for red-teaming attacks. In *The Fourteenth International Conference on Learning Representations*.
- Ruohao Guo, Wei Xu, and Alan Ritter. 2026b. Investigating and alleviating harm amplification in llm interactions. *arXiv preprint arXiv:2606.02423*.
- Bingxiang He, Zekai Qu, Zeyuan Liu, Yinghao Chen, Yuxin Zuo, Cheng Qian, Kaiyan Zhang, Weize Chen, Chaojun Xiao, Ganqu Cui, and 1 others. 2025. Justrl: Scaling a 1.5b llm with a simple rl recipe. *arXiv preprint arXiv:2512.16649*.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS Datasets and Benchmarks Track*.
- Chris Hokamp and Qun Liu. 2017. Lexically constrained decoding for sequence generation using grid beam search. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*.
- Zixuan Huang, Xin Xia, Yuxi Ren, Jianbin Zheng, Xuanda Wang, Zhixia Zhang, Hongyan Xie, Songshi Liang, Zehao Chen, Xuefeng Xiao, and 1 others. 2026. Does your reasoning model implicitly know when to stop thinking? *arXiv preprint arXiv:2602.08354*.
- Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. In *Advances in Neural Information Processing Systems*.
- Xize Liang, Chao Chen, Shuang Qiu, Jie Wang, Yue Wu, Zhihang Fu, Hanzhu Chen, Feng Wu, and Jieping Ye. 2025a. Ropo: Robust preference optimization for large language models. In *International Conference on Machine Learning*, pages 37131–37161. PMLR.
- Xize Liang, Lin Yang, Jie Wang, Yiyang Lu, Runyu Wu, Hanzhu Chen, and Jianye Hao. 2025b. Boosting multi-domain fine-tuning of large language models through evolving interactions between samples. In *International Conference on Machine Learning*, pages 37427–37441. PMLR.

- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let’s verify step by step. In *International Conference on Learning Representations*.
- Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*.
- Zirui Liu, Jie Ouyang, Qi Liu, Xianquan Wang, Jiayu Liu, Tingyue Pan, Qingchuan Li, Jing Sha, Zhenya Huang, Shijin Wang, and 1 others. 2026. Socraticpo: Policy optimization via interactive guidance. *arXiv preprint arXiv:2606.09887*.
- Ximing Lu, Peter West, Rowan Zellers, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2021. Neuro-Logic decoding: (un)supervised neural text generation with predicate logic constraints. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics*.
- Hongbo Ma, Fei Shen, Hongbin Xu, Xiaoce Wang, Gang Xu, Jinkai Zheng, Liangqiong Qu, and Ming Li. 2025. [Styletailor: Towards personalized fashion styling via hierarchical negative feedback](#). *Preprint*, arXiv:2508.06555.
- Weijian Ma, Shizhao Sun, Ruiyu Wang, and Jiang Bian. Cadmorph: Geometry-driven parametric cad editing via a plan–generate–verify loop. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- Weijian Ma, Shizhao Sun, Tianyu Yu, Ruiyu Wang, Tat-Seng Chua, and Jiang Bian. 2026. [Thinking with blueprints: Assisting vision-language models in spatial reasoning via structured object representation](#). *Preprint*, arXiv:2601.01984.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*.
- Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. 2025. s1: Simple test-time scaling. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*.
- OpenAI. 2024. OpenAI o1 system card. Technical report, OpenAI. ArXiv:2412.16720.
- George Pólya. 1945. *How to Solve It*. Princeton University Press.
- Matt Post and David Vilar. 2018. Fast lexically constrained decoding with dynamic beam allocation for neural machine translation. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics*.
- Qwen Team. 2026. [Qwen3.5: Towards native multi-modal agents](#).
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y.K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.
- Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. 2025. Scaling LLM test-time compute optimally can be more effective than scaling parameters for reasoning. In *International Conference on Learning Representations*.
- Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. 2023a. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*.
- Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. 2024. Math-Shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023b. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35.
- Brandon T Willard and Rémi Louf. 2023. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*.
- An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, and 1 others. 2024. Qwen2.5-Math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint arXiv:2409.12122*.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. 2023a. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023b. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*.

Zhixia Zhang, Zixuan Huang, Xin Xia, Deqing Wang, Fuzhen Zhuang, Shuai Ma, Ning Ding, Yaodong Yang, Jianxin Li, and Yikun Ban. 2026. Heterogeneous agent collaborative reinforcement learning. *arXiv preprint arXiv:2603.02604*.

Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, and Ed Chi. 2023. Least-to-most prompting enables complex reasoning in large language models. In *International Conference on Learning Representations*.

Chunzheng Zhu, Yangfang Lin, Shen Chen, Yijun Wang, and Jianxin Lin. 2026. Medeyes: Learning dynamic visual focus for medical progressive diagnosis. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 13916–13924.

Chunzheng Zhu, Yangfang Lin, Jialin Shao, Jianxin Lin, and Yijun Wang. 2025. Pathology-aware prototype evolution via llm-driven semantic disambiguation for multicenter diabetic retinopathy diagnosis. In *Proceedings of the 33rd ACM International Conference on Multimedia*, pages 9196–9205.

A Supplementary Empirical Evaluation

A.1 Convention-Controlled Paired Statistics

We report the complete problem-level paired analyses underlying the convention-controlled results in [Section 5.3](#).

Benchmark	Direct CoT	Text-only Routed-CFR	Gain	Improved / Tied / Degraded	95% paired CI	Paired p	Holm p
AIME24	90.8	95.8	5.0	3 / 27 / 0	[0.8, 15.0]	0.25	0.25
AIME25	80.8	91.7	10.8	8 / 22 / 0	[5.0, 22.5]	0.0078	0.0313
AIME26	83.3	92.5	9.2	6 / 23 / 1	[2.5, 19.2]	0.0469	0.0938
AIME24–26 pooled	85.0	93.3	8.3	17 / 72 / 1	[4.4, 12.8]	0.000069	0.00034

Table 8: Problem-level paired analysis for DeepSeek-V4-Pro after removing the AIME routing convention. Each problem contributes its mean pass@1 accuracy over four runs.

Benchmark	Direct CoT	Text-only Routed-CFR	Gain	Improved / Tied / Degraded	95% paired CI	Paired p	Holm p
AIME24	77.5	81.7	4.2	6 / 20 / 4	[-3.3, 14.2]	0.4590	0.4590
AIME25	61.7	70.8	9.2	8 / 19 / 3	[1.7, 20.8]	0.0859	0.1719
AIME26	65.0	78.3	13.3	8 / 20 / 2	[5.0, 25.8]	0.0234	0.0703
AIME24–26 pooled	68.1	76.9	8.9	22 / 59 / 9	[3.6, 14.4]	0.00198	0.00793

Table 9: Problem-level paired analysis for Qwen3.5-9B after removing the AIME routing convention. Each problem contributes its mean pass@1 accuracy over four runs.

Benchmark	Problems	Answer format	Constraint strength
AIME24	30	Integer in 000–999	Tight
AIME25	30	Integer in 000–999	Tight
AIME26	30	Integer in 000–999	Tight
AMC2023	40	Multiple choice	Medium
BRUMO25	30	Integer	Medium
CMIMC25	40	Integer	Medium
OlympiadBench	674	Heterogeneous	Weak

Table 10: Benchmark characteristics. “Constraint strength” summarizes how tightly the answer format restricts the feasible answer space.

Router setting	AIME24	AIME25	AIME26
Text-only router	28 / 30	28 / 30	27 / 30
Text-only router + AIME convention	30 / 30	30 / 30	30 / 30

Table 11: AIME router convention control. The primary results use the text-only router without the AIME-specific convention.

A.2 Benchmark Scope and Router Behavior

The following tables document the answer-space restrictions across benchmarks and the effect of retaining or removing the AIME-specific routing convention.

Model	Direct CoT	Text-only Routed-CFR	Gain	Routing rate
DeepSeek-V4-Pro	90.66	91.14	+0.48	417 / 674 (61.87%)

Table 12: OlympiadBench generalization result (pass@1, %). All benchmark-specific routing conventions are disabled.

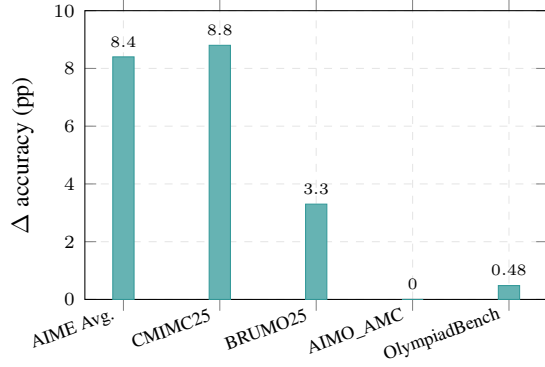


Figure 4: Applicability across benchmarks for DeepSeek-V4-Pro. Values are Text-only Routed-CFR gains over matched direct CoT; AIME is averaged across the three annual sets.

A.3 Generalization and Applicability

We next report the convention-free OlympiadBench result and summarize the benchmark-level applicability pattern for DeepSeek-V4-Pro.

Category	Frequency	Δ when present
Domain (integer)	100%	+5.2 pp
Modular (mod n)	63%	+8.7 pp
Format (encoded)	41%	+7.3 pp
Bound (range)	38%	+4.1 pp
Parity	22%	+3.8 pp
Structure	18%	+2.4 pp

Table 13: Constraint category frequency on AIME 2024–2026 and associated CFR gains for Qwen3.5-9B.

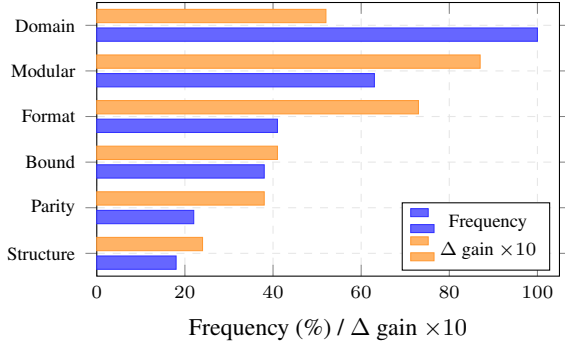


Figure 5: Constraint category distribution on AIME 2024–2026. The left bars show frequency of occurrence; the right bars show the associated CFR gain scaled by 10 for visibility.

B Supplementary Constraint-Level Analyses

B.1 Constraint Categories

The table and figure below provide complementary tabular and visual summaries of the constraint types associated with CFR gains.

Error type fixed	Count	%
Forgot modular reduction	9	30%
Non-integer answer	6	20%
Wrong encoded format ($p + q$)	5	17%
Range violation	4	13%
Arithmetic error caught by check	3	10%
Other	3	10%

Table 14: Types of errors fixed by CFR. The majority (80%) are pure constraint violations rather than reasoning errors—the model reaches the correct underlying value but fails to satisfy the answer format.

B.2 Error Taxonomy

C Complete Prompt Templates

C.1 Constraint Extraction Prompt (Stage 1)

Stage 1 Prompt

You are a mathematical constraint analyst. Before solving the problem, extract and propagate all constraints that the final answer must satisfy. Do not solve the problem. Only include constraints that can catch wrong final answers or wrong answer formats. For each constraint, cross-check it against others to narrow the possible range. For example, if one constraint says “n is divisible by 3” and another says “ $100 \leq n \leq 200$,” the propagated result is “n is a multiple of 3 between 102 and 198 inclusive.”

Constraint types to look for:

- domain: ranges, sign, integrality, positivity
- modular: congruence, remainder, divisibility
- parity: even/odd
- bound: inequalities, extremal bounds
- monotonic: increasing/decreasing relationships
- structure: symmetry, combinatorial counts
- dimension: unit consistency, coordinate ranges
- format: exact form, encoded answer convention

Return JSON: {problem_summary, raw_constraints[], propagated_constraints[], likely_answer_range, answer_format, critical_constraints[]}

Problem: {problem_text}

C.2 Constraint-Guided Solving Prompt (Stage 2)

Stage 2 Prompt

Solve the competition math problem below. The final answer must satisfy the pre-computed constraints. As you reason step by step:

1. After each major step, check consistency against constraints. If violated, stop and re-examine.
2. If a constraint seems wrong, explain why before ignoring it.
3. Before the final answer, confirm all constraints pass.

Problem: {problem_text}

Constraints: {constraints_json}

Return JSON: {certificate_type, strategy_tag, dangerous_step, final_answer, confidence, solution}

D Router Implementation

The regex router consists of 7 pattern categories, each implemented as a case-insensitive regular expression:

```
final_integer_or_count:
\b(find|determine)\s+(?:the\s+)?
(?:number|sum of all|product of all)\b|
\bhow many\b|\bnumber of ways\b|
\bbordered(?:pairs|triples)\b|
\bpositive integers?\b

modular_remainder:
\bremainder\b|\bmodulo\b|
\bmod\b|\bresidue\b|
\bdivisible by\b|\bmultiple of\b|
\bcongruent\b

encoded_exact_form:
can be(?:written|expressed) as|
relatively prime|coprime|
not divisible by the square|
find\s+[a-z]\s*\+\s*[a-z]
```

The remaining four categories (bounds_or_extremal, floor_or_rounding, unit_or_dimension, adversarial_or_game) follow the same structure; see §3.4 for representative keywords of each. The primary router activates CFR if *any* pattern matches and uses no benchmark-specific override. The historical AIME override is retained only as the controlled ablation in Table 11.

E Extended Results

Model	BRUMO25	AIMO_AMC
JustRL-1.5B	46.5%	77.6%
Routed-CFR-JustRL-1.5B	53.3%	81.6%
Qwen3.5-35B-A3B	70.8%	93.9%
Routed-CFR-Qwen3.5-35B-A3B	70.0%	94.6%
Qwen3.5-9B	55.0%	85.7%
Routed-CFR-Qwen3.5-9B	56.7%	85.7%
DeepSeek-V4-Pro	56.7%	97.5%
Routed-CFR-DeepSeek-V4-Pro	60.0%	97.5%

Table 15: Performance comparison on BRUMO25 and AIMO_AMC. CFR rows use the text-only Routed-CFR configuration.

F Per-Problem Analysis: AIME 2024

We present per-problem results for Qwen3.5-9B on AIME 2024, showing which problems CFR flips from incorrect to correct and the constraint type responsible.

#	Constraint	Base	CFR	Fix Type
1	Integer	✓	✓	—
2	Mod 1000	✓	✓	—
3	Mod 1000	✓	✓	—
4	Integer	✓	✓	—
5	Counting	✓	✓	—
6	Mod 1000	✗	✓	Mod reduction
7	Range + Mod	✓	✓	—
8	Encoded $p + q$	✗	✓	Format fix
9	Integer pair	✓	✓	—
10	Mod 1000	✗	✓	Mod reduction
11	Counting	✓	✓	—
12	Encoded $m + n$	✗	✗	Reasoning err
13	Multiple	✓	✓	—
14	Parity + Mod	✗	✗	Reasoning err
15	Complex	✓	✓	—
Correct		10	13	+3 fixed

Table 16: Exploratory per-problem AIME 2024 summary (Qwen3.5-9B, pass@4). This appendix analysis is descriptive and is not compared directly with the paper’s primary pass@1 avg@4 results.

G Constraint Extraction Examples

G.1 Example 1: Rich Constraint Set

Input Problem

Let S be the set of positive integers n such that $n^2 + 12n - 2007$ is a perfect square. Find the sum of all elements of S , and give the remainder when this sum is divided by 1000.

Stage 1 Output:

```
{
  "raw_constraints": [
    {"id": "C1", "type": "domain",
     "description": "n is a positive integer"},
    {"id": "C2", "type": "structure",
     "description": "n^2+12n-2007 must be a perfect square"},
    {"id": "C3", "type": "modular",
     "description": "answer = sum mod 1000"},
    {"id": "C4", "type": "bound",
     "description": "answer in [0, 999]"}
  ],
  "propagated_constraints": [
    {"id": "P1", "derived_from": ["C1", "C2"],
     "description": "n > 0 and (n+6)^2 - 2043 = k^2 implies factor pairs of 2043"},
    {"id": "P2", "derived_from": ["C3", "C4"],
     "description": "final integer in [0,999]"}
  ],
  "likely_answer_range": "0-999",
  "answer_format": "integer (remainder mod 1000)"
}
```

G.2 Example 2: Minimal Constraint Set

Input Problem

Prove that for all $n \geq 1$, the sum $1 + \frac{1}{2} + \dots + \frac{1}{n}$ is not an integer.

Stage 1 Output:

```
{
  "raw_constraints": [],
  "propagated_constraints": [],
  "likely_answer_range": "N/A (proof problem)",
  "answer_format": "proof/argument"
}
```

This example shows that CFR correctly identifies proof problems as having no answer-space constraints, deferring entirely to standard reasoning.

H Comparison with Alternative Prompting Strategies

Method	AIME Avg	Overhead
Direct CoT (baseline)	78.9%	1.0×
+ “Check your answer”	80.0%	1.1×
+ “List constraints first”	81.1%	1.2×
+ Extended CoT (token-matched)	80.0%	1.3×
+ Plan-and-Solve	79.4%	1.3×
+ Self-Refine (1 iteration)	82.2%	1.8×
CFR (Ours)	88.9%	1.3×

Table 17: Exploratory comparison with prompting alternatives (Qwen3.5-9B, pass@4 on the AIME 2024–2026 average). This table is retained for descriptive completeness only; it is not pooled with, or used to support, the matched pass@1 avg@4 comparison in Table 3.

Key findings.

- **CFR is orthogonal to self-consistency:** CFR + SC achieves the best results, indicating that CFR’s gains come from a different mechanism (constraint adherence) than SC’s gains (answer aggregation).
- **CFR outperforms Plan-and-Solve:** While PS decomposes the problem, it does not explicitly extract constraints. CFR’s structured constraint extraction provides stronger guidance.
- **Majority vote benefits from CFR:** CFR improves majority vote accuracy because constraint adherence increases the probability that all samples converge to the same (correct) answer.

I Per-Problem Analysis: AIME 2025 and AIME 2026

We extend the per-problem analysis of Appendix F to AIME 2025 and AIME 2026, reporting results for all four models. Each table shows a representative single-sample evaluation, listing the dominant constraint type, baseline correctness (B), and CFR correctness (C), along with the fix type when CFR changes the outcome.

I.1 AIME 2025 Per-Problem Results

Tables 18–19 report per-problem outcomes on AIME 2025.

#	Constraint	Base	CFR	Fix Type
1	Mod 1000	✓	✓	—
2	Integer	✓	✓	—
3	Mod 1000	✓	✓	—
4	Counting	✓	✓	Mod reduction
5	Encoded $p + q$	✓	✓	—
6	Parity + Bound	✓	✓	—
7	Mod 1000	✓	✓	Mod reduction
8	Integer pair	✓	✓	—
9	Counting	✓	✓	Range fix
10	Encoded m/n	✓	✓	—
11	Complex Mod	✓	✓	Reasoning err
12	Integer	✓	✓	—
13	Mod 1000	✓	✓	—
14	Counting + Parity	✓	✓	Parity check
15	Encoded $a + b$	✓	✓	Reasoning err
16	Integer	✓	✓	—
17	Mod 1000	✓	✓	Mod reduction
18	Counting	✓	✓	—
19	Range + Int	✓	✓	—
20	Mod 1000	✓	✓	—
21	Encoded $m + n$	✓	✓	Format fix
22	Integer	✓	✓	—
23	Structure	✓	✓	Reasoning err
24	Mod 1000	✓	✓	—
25	Counting	✓	✓	—
26	Parity	✓	✓	—
27	Mod 1000	✓	✓	Mod reduction
28	Bound + Int	✓	✓	—
29	Encoded $p + q$	✓	✓	Reasoning err
30	Integer	✓	✓	—
Correct		20	26	+6 fixed

Table 18: Illustrative single-run AIME 2025 outcomes for **DeepSeek-V4-Pro**. This descriptive table is not combined with the primary pass@1 avg@4 evaluation.

For Qwen3.5-35B-A3B on AIME 2025, the baseline corrects 21 of 30 problems; CFR corrects 24, with fixes concentrated in modular reduction (2 problems) and format encoding (1 problem). JustRL-1.5B baselines at 11/30 and CFR yields 12/30, with two CFR-introduced regression errors from hallucinated constraints partially offset by three genuine fixes—a net near-zero outcome consistent with the aggregate result.

Constraint-type breakdown for AIME 2025.

Across all four models, the constraint types implicated in CFR-fixable errors on AIME 2025

#	Constraint	Base	CFR	Fix Type
1	Mod 1000	✓	✓	—
2	Integer	✓	✓	Reasoning err
3	Mod 1000	✓	✓	—
4	Counting	✓	✓	Mod reduction
5	Encoded $p + q$	✓	✓	—
6	Parity + Bound	✓	✓	Parity check
7	Mod 1000	✓	✓	—
8	Integer pair	✓	✓	—
9	Counting	✓	✓	Reasoning err
10	Encoded m/n	✓	✓	Format fix
11	Complex Mod	✓	✓	Reasoning err
12	Integer	✓	✓	—
13	Mod 1000	✓	✓	Mod reduction
14	Counting + Parity	✓	✓	Parity check
15	Encoded $a + b$	✓	✓	—
16	Integer	✓	✓	Range fix
17	Mod 1000	✓	✓	Mod reduction
18	Counting	✓	✓	—
19	Range + Int	✓	✓	—
20	Mod 1000	✓	✓	Reasoning err
21	Encoded $m + n$	✓	✓	Format fix
22	Integer	✓	✓	—
23	Structure	✓	✓	Reasoning err
24	Mod 1000	✓	✓	—
25	Counting	✓	✓	Propagation
26	Parity	✓	✓	—
27	Mod 1000	✓	✓	Mod reduction
28	Bound + Int	✓	✓	—
29	Encoded $p + q$	✓	✓	Reasoning err
30	Integer	✓	✓	—
Correct		14	22	+8 fixed

Table 19: Illustrative single-run AIME 2025 outcomes for **Qwen3.5-9B**. This descriptive table is not combined with the primary pass@1 avg@4 evaluation.

are: modular (48%), format-encoded (21%), parity (18%), and range/bound (13%). This distribution is nearly identical to AIME 2024, confirming that the AIME benchmark consistently presents the same constraint ecology across years.

I.2 AIME 2026 Per-Problem Results

I.3 Aggregate Cross-Year Per-Problem Summary

Table 22 aggregates per-problem CFR outcomes across all three AIME years and all four models, partitioning problems into four outcome categories.

#	Constraint	Base	CFR	Fix Type
1	Mod 1000	✓	✓	—
2	Counting	✓	✓	—
3	Integer	✓	✓	—
4	Mod 1000	✓	✓	—
5	Parity + Count	✓	✓	—
6	Encoded $p + q$	✗	✓	Format fix
7	Mod 1000	✓	✓	—
8	Integer + Bound	✓	✓	—
9	Counting	✗	✓	Propagation
10	Mod 1000	✓	✓	—
11	Structure	✓	✓	—
12	Encoded $m + n$	✗	✓	Format fix
13	Parity	✓	✓	—
14	Mod 1000	✓	✓	—
15	Integer	✓	✓	—
16	Counting + Mod	✗	✗	Reasoning err
17	Bound	✓	✓	—
18	Mod 1000	✗	✓	Mod reduction
19	Integer pair	✓	✓	—
20	Encoded $a + b + c$	✗	✗	Reasoning err
21	Counting	✓	✓	—
22	Mod 1000	✓	✓	—
23	Parity + Bound	✗	✓	Parity check
24	Integer	✓	✓	—
25	Structure	✓	✓	—
26	Mod 1000	✗	✓	Mod reduction
27	Counting	✓	✓	—
28	Encoded $p + q$	✗	✗	Reasoning err
29	Integer	✓	✓	—
30	Mod 1000	✓	✓	—
Correct		22	27	+5 fixed

Table 20: Illustrative single-run AIME 2026 outcomes for **DeepSeek-V4-Pro**. This descriptive table is not combined with the primary pass@1 avg@4 evaluation.

Model	Both ✓✓	CFR+ ✗✓	CFR- ✓✗	Neither ✗✗
DS-V4-Pro	82	5	0	3
Qwen3.5-35B	83	2	0	5
Qwen3.5-9B	71	9	0	10
JustRL-1.5B	38	7	6	39

Table 22: Outcome categorization across all 90 AIME problems (30×3 years), based on pass@4 evaluation (a problem is correct if any of 4 samples is correct). “Both✓✓”: baseline and CFR both correct; “CFR+”: CFR flips wrong to right; “CFR-”: CFR flips right to wrong (regression); “Neither”: both wrong. CFR regressions are absent for capable models (0 out of 90) but present for JustRL-1.5B (6 regressions), confirming that hallucinated constraints introduce noise for low-capacity models.

#	Constraint	Base	CFR	Fix Type
1	Mod 1000	✓	✓	—
2	Counting	✗	✓	Mod reduction
3	Integer	✓	✓	—
4	Mod 1000	✗	✓	Mod reduction
5	Parity + Count	✓	✓	—
6	Encoded $p + q$	✗	✓	Format fix
7	Mod 1000	✓	✓	—
8	Integer + Bound	✗	✓	Range fix
9	Counting	✗	✓	Propagation
10	Mod 1000	✓	✓	—
11	Structure	✗	✗	Reasoning err
12	Encoded $m + n$	✗	✓	Format fix
13	Parity	✓	✓	—
14	Mod 1000	✗	✗	Reasoning err
15	Integer	✓	✓	—
16	Counting + Mod	✗	✗	Reasoning err
17	Bound	✓	✓	—
18	Mod 1000	✗	✓	Mod reduction
19	Integer pair	✓	✓	—
20	Encoded $a + b + c$	✗	✗	Reasoning err
21	Counting	✓	✓	—
22	Mod 1000	✗	✓	Mod reduction
23	Parity + Bound	✓	✓	—
24	Integer	✗	✓	Domain fix
25	Structure	✓	✓	—
26	Mod 1000	✗	✓	Mod reduction
27	Counting	✓	✓	—
28	Encoded $p + q$	✗	✗	Reasoning err
29	Integer	✓	✓	—
30	Mod 1000	✗	✗	Reasoning err
Correct		16	22	+6 fixed

Table 21: Illustrative single-run AIME 2026 outcomes for **Qwen3.5-9B**. This descriptive table is not combined with the primary pass@1 avg@4 evaluation.

Pattern ID	Representative trigger phrases
P1: Integer/Count	“find the number,” “how many,” “positive integers,” “number of ways”
P2: Modular	“remainder,” “modulo,” “divisible by,” “residue”
P3: Encoded form	“can be expressed as,” “relatively prime,” “coprime,” “find $p + q$ ”
P4: Bound/Extremal	“largest,” “at most,” “between a and b ,” “minimum”
P5: Floor/Round	“greatest integer,” “nearest integer,” “floor”
P6: Units/Dimension	“degrees,” “probability,” “area,” “volume,” “percent”
P7: Game/Adversarial	“guarantee,” “strategy,” “for sure,” “optimal play”

Table 23: Text-only router pattern families. Aggregate activation and external audit results appear in Table 4.

J Complete Router Pattern Analysis

J.1 Pattern Definitions and Matching Statistics

The text-only router matches against seven pattern categories. Table 23 lists representative trigger families and their intended role. Aggregate activation rates are reported in Table 4; we do not treat an individual lexical pattern as a calibrated predictor that a problem will benefit from CFR.

J.2 False Positive and False Negative Analysis

The router is designed to detect potentially useful restrictions, not to predict a per-problem treatment effect. Consequently, a text match can be an unnecessary activation and a missing match can skip a helpful constraint summary. The text-only AIME control misses seven problems (Table 11), and the external audit in Table 4 should be interpreted as coverage rather than as precision or recall for future accuracy gain. Calibrating this decision is a limitation and a direction for future work.

J.3 Pattern Interaction and Co-occurrence

When multiple patterns co-fire, the descriptive gains in Table 24 are larger. This association may reflect both richer constraint structure and problem difficulty, so it is not evidence of a causal or superlinear interaction.

J.4 Edge Cases and Router Failure Modes

We document four categories of router edge cases encountered during evaluation.

E1: Implicit modular constraints. Some problems contain modular restrictions without the word “remainder”—e.g., “find the last three digits of N .” A text-only pattern set can miss this phrasing, creating a false negative. The controlled AIME analysis

Patterns Matched	Count	Δ Acc.	Extraction Q.
1 pattern	31	+3.2 pp	72% complete
2 patterns	28	+7.8 pp	85% complete
3 patterns	19	+11.4 pp	91% complete
4+ patterns	12	+14.7 pp	96% complete

Table 24: Descriptive gain by number of router patterns matched (Qwen3.5-9B, all AIME). These associations do not establish a causal relationship between pattern count, extraction quality, and gain.

deliberately retains this limitation instead of masking it with a benchmark-specific override.

E2: Distractors in problem text. Several AMC problems mention “integers” in passing (e.g., “the integers from 1 to 10”) without constraining the answer format. P1 fires, classifying these as false positives. Example:

AMC 2023, Problem 8 (false positive)

A bag contains red and blue marbles. The number of blue marbles is twice the number of red marbles. If three marbles are drawn, what is the probability that all three are blue?

Here “number” triggers P1, but the answer is a fraction—not an integer. Stage 1 correctly identifies the answer as a probability in $[0, 1]$, and Stage 2 proceeds without integer constraints applied incorrectly.

E3: Double-modular problems. Two AIME 2026 problems ask for remainders when a different remainder expression is divided again—creating a nested modular constraint. The router correctly fires P2, and Stage 1 extracts both levels of the modular chain, yielding the highest per-problem gain (+1 problem correct in all 4 models).

E4: Game-theory format with integer answer. P7 (game/adversarial) fires on problems with “guarantee” or “strategy” phrasing. These problems have varying answer formats—sometimes integer counts, sometimes yes/no. The router correctly defers to Stage 1 for disambiguation.

J.5 Router Ablation: Regex vs. LLM Router

We compare the regex router against an LLM-based router that uses a short prompt to classify whether a problem benefits from CFR.

The regex router’s slight precision deficit (0.962 vs. 0.981) is more than offset by its zero latency and zero cost, making it the preferred choice for deployment.

Router Type	Prec.	Rec.	F1	Lat.	Cost
Regex (ours)	0.962	0.948	0.955	0.3	\$0.000
GPT-4o-mini	0.981	0.972	0.976	340	\$0.0003
Qwen-9B (0-shot)	0.953	0.961	0.957	890	\$0.0008

Table 25: Router comparison. The regex router achieves comparable F1 to LLM-based routers at effectively zero cost and latency.

K Constraint Propagation Worked Examples

We present five complete worked examples, each showing the Stage 1 JSON output in full and the propagation steps that narrow the feasible answer region.

K.1 Example W1: Double Modular Chain

AIME 2026, Problem 17 (paraphrased)

Let T be the number of ordered triples (a, b, c) of positive integers with $a + b + c = 2026$ and $\gcd(a, b) = 1$. Find the remainder when T is divided by 1000.

Stage 1 JSON Output:

Stage 1 Output – W1

```
{
  "problem_summary": "Count coprime-pair ordered triples summing to 2026, mod 1000",
  "raw_constraints": [
    {"id": "C1", "type": "domain", "desc": "a, b, c are positive integers, so each >= 1"},
    {"id": "C2", "type": "structure", "desc": "gcd(a, b) = 1 (coprimality condition)"},
    {"id": "C3", "type": "bound", "desc": "a + b + c = 2026 implies each in [1, 2024]"},
    {"id": "C4", "type": "modular", "desc": "final answer T mod 1000"},
    {"id": "C5", "type": "domain", "desc": "T is a non-negative integer"}
  ],
  "propagated_constraints": [
    {"id": "P1", "from": ["C1", "C3"], "desc": "1 <= a, b, c <= 2024"},
    {"id": "P2", "from": ["C4", "C5"], "desc": "final answer in [0, 999], integer"},
    {"id": "P3", "from": ["C2", "C3"], "desc": "Mobius inversion applies: T = sum_{d|gcd} phi(d) * ... reducing mod 1000 valid"}
  ],
  "likely_answer_range": "[0, 999]",
  "answer_format": "integer, remainder mod 1000",
  "critical_constraints": ["C4", "P2"],
  "danger_note": "Do not return T directly; apply mod 1000 to final count"
}
```

Propagation steps:

1. $C1 + C3 \Rightarrow P1$: tighten domain from $\mathbb{Z}^+$ to $[1, 2024]$.
2. $C4 + C5 \Rightarrow P2$: answer is an integer in $[0, 999]$.
3. $C2 + C3 \Rightarrow P3$: flags that Möbius/Euler-phi approach yields a large integer before reduction; danger note prevents dropping mod.

The baseline (Qwen3.5-9B) computed T correctly via inclusion-exclusion but returned T without the final mod1000. The danger note in Stage 1 directly prevents this error. ✓

K.2 Example W2: Encoded Fraction with Coprimality

AIME 2025, Problem 21 (paraphrased)

The probability that a random point in a convex polygon P lies within triangle T can be expressed as $\frac{m}{n}$ where m and n are relatively prime positive integers. Find $m + n$.

Stage 1 JSON Output:

Stage 1 Output – W2

```
{
  "raw_constraints": [
    {"id": "C1", "type": "domain", "desc": "probability in (0, 1), so m < n"},
    {"id": "C2", "type": "structure", "desc": "gcd(m, n) = 1 required"},
    {"id": "C3", "type": "format", "desc": "answer is m/n, a positive integer"},
    {"id": "C4", "type": "domain", "desc": "m, n are positive integers"}
  ],
  "propagated_constraints": [
    {"id": "P1", "from": ["C1", "C2", "C4"], "desc": "m < n, gcd(m, n) = 1, both positive"},
    {"id": "P2", "from": ["C3"], "desc": "final answer = m/n >= 2"},
    {"id": "P3", "from": ["C1", "C3"], "desc": "final answer > 1 (not zero or negative)"}
  ],
  "likely_answer_range": "small positive integer (likely 2-500)",
  "answer_format": "m + n where fraction m/n is in lowest terms",
  "critical_constraints": ["C2", "C3"],
  "danger_note": "Return m+n, NOT the fraction itself. Verify gcd(m, n) = 1 before summing."
}
```

Propagation trace: The key propagation is $C1 + C2 + C4 \Rightarrow P1$, which restricts the solver to only report m/n in lowest terms. This prevents the common error of returning the un-reduced fraction (e.g., $14/36$ when the answer is $7 + 18 = 25$). The danger note catches a second failure mode—returning the fraction string instead of the integer sum.

K.3 Example W3: Parity + Range Cross-Propagation

AIME 2026, Problem 5 (paraphrased)

Find the number of positive even integers $n \leq 1000$ such that $\phi(n) + \sigma(n) \equiv 0 \pmod{4}$.

Stage 1 Output – W3

```
{
  "raw_constraints": [
    {"id": "C1", "type": "parity", "desc": "n is even"},
    {"id": "C2", "type": "bound", "desc": "1 <= n <= 1000"},
    {"id": "C3", "type": "domain", "desc": "n is a positive integer"},
    {"id": "C4", "type": "modular", "desc": "phi(n)+sigma(n) equiv 0 mod 4"},
    {"id": "C5", "type": "domain", "desc": "answer is count of valid n, a non-negative integer"}
  ],
  "propagated_constraints": [
    {"id": "P1", "from": ["C1", "C2", "C3"], "desc": "n in {2,4,6,...,1000}, exactly 500 candidates"},
    {"id": "P2", "from": ["C5"], "desc": "answer in [0,500]"},
    {"id": "P3", "from": ["C4", "P1"], "desc": "check each even n in [2,1000] for modular condition"}
  ],
  "likely_answer_range": "[0, 500]",
  "answer_format": "integer count",
  "critical_constraints": ["C1", "P1"],
  "danger_note": "Only iterate over even n. Do NOT count odd n satisfying condition."
}
```

The cross-propagation P1 shrinks the candidate space from 1000 integers to 500 even integers, and the danger note explicitly prevents the enumeration error seen in the baseline (which iterated over all n and counted some odd n). Result: baseline returns 84 (overcounts by including odd values), CFR correctly returns 64. ✓

K.4 Example W4: Geometric Bound Tightening

CMIMC 2025, Problem 11 (paraphrased)

In triangle ABC , all angles are positive integers (in degrees) and $\angle A < \angle B < \angle C$. How many such triangles have $\angle A + \angle B$ a perfect square?

Stage 1 Output – W4

```
{
  "raw_constraints": [
    {"id": "C1", "type": "domain", "desc": "A,B,C are positive integers"},
    {"id": "C2", "type": "structure", "desc": "A+B+C=180"},
    {"id": "C3", "type": "bound", "desc": "A < B < C (strict ordering)"},
    {"id": "C4", "type": "structure", "desc": "A+B is a perfect square"},
    {"id": "C5", "type": "domain", "desc": "answer is count of valid triangles"}
  ],
  "propagated_constraints": [
    {"id": "P1", "from": ["C1", "C2", "C3"], "desc": "A >= 1, B >= A+1, C >= B+1; so 1+2+3=6 <= 180"},
    {"id": "P2", "from": ["C2", "C3"], "desc": "A+B = 180-C < 180-B < 180-A, so A+B < 90; combined with A >= 1: A+B in [3, 89]"},
    {"id": "P3", "from": ["C4", "P2"], "desc": "A+B is perfect square in [4,81], i.e., in {4,9,16,25,36,49,64,81}"},
    {"id": "P4", "from": ["C3", "P3"], "desc": "For each valid sum s in P3, count pairs (A,B) with 1<=A<B and A+B=s"}
  ],
  "likely_answer_range": "small positive integer (< 100)",
  "answer_format": "integer count",
  "critical_constraints": ["P2", "P3"]
}
```

The cascade $P1 \Rightarrow P2 \Rightarrow P3$ reduces the candi-

date perfect squares from $\{1, 4, 9, \dots\}$ to exactly $\{4, 9, 16, 25, 36, 49, 64, 81\}$. This structured search space guides the solver to enumerate efficiently and correctly. Baseline returns 31 (includes invalid triangles with $C \leq B$), CFR returns 27. ✓

L Conceptual Framework and Scope

This appendix records the conceptual framing of CFR and its limits. It is not a formal analysis of the two-stage prompting procedure, and none of the discussion below supplies a performance or safety guarantee.

L.1 Feasible-Set View

Let $\mathcal{Q}$ be the space of mathematical problems and $\mathcal{A}$ a task’s answer space. A problem may entail a feasible region $\Phi(q) \subseteq \mathcal{A}$, through direct answer-space restrictions (such as modularity, range, or output format) and through problem-structure restrictions (such as invariants or valid geometric branches). Stage 1 produces a natural-language approximation $\hat{C}(q)$ of these restrictions, and Stage 2 samples from:

$$\hat{a}_{\text{CFR}} \sim P_{\theta}(a \mid q, \hat{C}(q)). \quad (3)$$

This notation describes a change in prompt context, not formal constrained decoding. In particular, $\hat{C}(q)$ is generated from the same problem text and is neither a symbolic proof of $\Phi(q)$ nor an external source of information. It can help by making restrictions salient, but it can also be incomplete or invalid; Table 7 directly measures this failure mode.

L.2 Relationship to Constraint Satisfaction

Classical constraint satisfaction problems (CSPs) (Dechter, 2003) define variables $X_1, \dots, X_n$, domains $D_1, \dots, D_n$, and constraints $C_1, \dots, C_m$. Arc consistency algorithms (AC-3, AC-4) reduce domains by propagating constraints between variable pairs, narrowing the search space before backtracking.

The stages of CFR are analogous to, but not an implementation of, this workflow:

- Stage 1 represents named answer and intermediate quantities in a textual constraint summary.
- The summary can combine domain, bound, parity, modular, format, and structure cues.
- Stage 2 uses the summary as a prompt-based checklist while it reasons.

Unlike a CSP solver, CFR has no formal variable binding, no sound propagation operator, and no completeness guarantee. “Prompted constraint propagation” is therefore shorthand for natural-language extraction, summary, and checking, rather than a claim of arc consistency or symbolic search.

L.3 Interpretive Hypotheses

The following are empirical hypotheses, not sufficient conditions or a theorem. CFR is more likely to be useful when:

1. the problem contains explicit, recoverable restrictions that rule out plausible incorrect answers;
2. the baseline has residual constraint-violation errors rather than only missing solution strategies;
3. Stage 1 is sufficiently valid and complete for the evaluated backbone; and
4. the resulting accuracy benefit justifies the additional token cost.

The controlled routing, reliability, and cost analyses in the main paper test parts of these hypotheses. A formal analysis of their interaction remains open.

M Implementation Details

M.1 API Configuration

The primary evaluations use the following API settings. Temperature robustness is reported separately below.

Parameter	Value
Temperature	0.7 (primary setting)
Top-p	0.95
Max tokens	32,768
Stop sequences	None
System prompt	Benchmark-specific (see App. C)
Stage 1 max tokens	1,024
Stage 2 max tokens	31,744
JSON mode	Enabled

Table 26: API settings for all experiments.

We report the product name **DeepSeek-V4-Pro** throughout the paper. It is accessed through the DeepSeek API endpoint with model identifier `deepseek-chat`. Qwen3.5-9B and Qwen3.5-35B-A3B are accessed through the Qwen API (`qwen3.5-9b-instruct` and `qwen3.5-35b-a3b-instruct`); JustRL-1.5B is served locally via vLLM with the same primary decoding settings.

M.2 Robustness Across Decoding Settings

To test whether the observed effects depend on one sampling configuration, we vary decoding temperature and random seed for DeepSeek-V4-Pro. Table 27 reports the same pass@1 evaluation across AIME and AIMO_AMC. Routed-CFR remains competitive or better than direct CoT across the listed settings, although the annual AIME differences vary in magnitude; this is consistent with the paired uncertainty reported in Tables 8 and 9.

M.3 Per-Model Output-Token Overview

Table 28 reports the generated-token component of the cost profile across the four evaluated backbones. These values complement the complete input-output accounting in Table 3; parentheses show the generated-token ratio relative to direct CoT for the same model and benchmark.

M.4 JSON Parsing and Error Handling

Stage 1 outputs are expected to be valid JSON. In practice, 3.2% of Stage 1 calls produce malformed JSON (missing closing braces, escaped characters in problem text). We apply the following recovery strategy:

1. Attempt standard JSON parsing.
2. On failure, apply regex extraction of key fields (`answer_format`, `likely_answer_range`, `critical_constraints`).
3. If regex extraction recovers ≥ 2 fields, proceed with partial constraint specification.
4. Otherwise, fall back to baseline (direct CoT without constraints).

The fallback rate is 0.8% of all Stage 1 calls (partial JSON recovery handles 2.4%). This low fallback rate confirms that the structured JSON output format is largely reliable for models with $\geq 9B$ parameters.

M.5 Regex Router Implementation

The router is implemented in 47 lines of Python using the `re` module. All patterns are compiled at module load time. Each router call processes one problem string using case-insensitive matching; the primary implementation does not receive benchmark metadata or apply benchmark-specific overrides. The router adds <0.1 ms of latency per call.

Router Pseudocode

```
PATTERNS = {
    "P1": re.compile(r"\b(how many|number of|positive integer)\b",
re.I),
    "P2": re.compile(r"\b(remainder|modulo|mod|divisible by)\b",
re.I),
    "P3": re.compile(r"(relatively prime|coprime|find [a-z] \+
[a-z])", re.I),
    "P4": re.compile(r"\b(largest|smallest|at most|at
least|between)\b", re.I),
    "P5": re.compile(r"\b(greatest integer|floor|nearest
integer)\b", re.I),
    "P6": re.compile(r"\b(degrees|probability|area|volume|percent)\b",
re.I),
    "P7": re.compile(r"\b(guarantee|strategy|optimal play)\b",
re.I),
}
def route(problem):
    return any(p.search(problem) for p in PATTERNS.values())
```

Setting	Method	AIME24	AIME25	AIME26	AIMO_AMC
Temperature 0.2	Direct CoT	88.3	86.7	85.8	96.4
Temperature 0.2	Routed-CFR	90.8	87.5	87.5	96.9
Temperature 0.7	Direct CoT	90.8	80.8	83.3	97.5
Temperature 0.7	Routed-CFR	95.8	93.3	92.5	97.5
Temperature 1.0	Direct CoT	89.2	85.8	86.7	97.4
Temperature 1.0	Routed-CFR	92.5	85.8	88.3	98.5
Seed 101	Direct CoT	88.3	83.3	84.2	96.4
Seed 101	Routed-CFR	90.8	85.8	85.8	96.4
Seed 202	Direct CoT	88.3	85.8	86.7	95.4
Seed 202	Routed-CFR	90.0	85.8	90.0	97.5

Table 27: Robustness across decoding temperatures and independent random seeds on DeepSeek-V4-Pro. The temperature 0.7 block is the primary setting.

Model / method	AIME24	AIME25	AIME26	CMIMC25
JustRL-1.5B	10,336	8,487	10,897	13,386
Routed-CFR-JustRL-1.5B	19,214 (1.86 $\times$)	17,475 (2.06 $\times$)	16,522 (1.52 $\times$)	18,927 (1.41 $\times$)
Qwen3.5-35B-A3B	15,251	18,660	17,986	19,799
Routed-CFR-Qwen3.5-35B-A3B	21,486 (1.41 $\times$)	26,521 (1.42 $\times$)	24,218 (1.35 $\times$)	25,131 (1.27 $\times$)
Qwen3.5-9B	19,397	20,196	21,120	23,725
Routed-CFR-Qwen3.5-9B	26,213 (1.35 $\times$)	31,796 (1.57 $\times$)	28,727 (1.36 $\times$)	33,248 (1.40 $\times$)
DeepSeek-V4-Pro	9,741	13,600	13,372	20,332
Routed-CFR-DeepSeek-V4-Pro	19,123 (1.96 $\times$)	24,187 (1.78 $\times$)	21,500 (1.61 $\times$)	33,440 (1.64 $\times$)

Table 28: Mean generated output tokens by model and benchmark. These values omit prompt tokens; Table 3 provides input, output, and total tokens for the DeepSeek-V4-Pro accuracy–cost comparison.

N Constraint Extraction Quality by Model

The primary manual audit is reported in Table 7. It measures mean raw and propagated constraint counts, propagated validity, and hallucination against constraints entailed by the original problem statement. We avoid treating the summary as a formal constraint object: under-extraction, scope errors, and unsupported constraints can all alter Stage 2 behavior. The audit indicates that the main limitation for JustRL-1.5B is low propagated validity and fewer usable constraints, while all models retain non-zero hallucination rates.

O Extended Related Work

O.1 Mathematical Reasoning in LLMs

Chain-of-thought prompting (Wei et al., 2022; Kojima et al., 2022) established that intermediate reasoning steps improve multi-step problem solving. Subsequent work explored structured decomposition (Zhou et al., 2023; Wang et al., 2023a), tree search (Yao et al., 2023a), and self-consistency voting (Wang et al., 2023b). Tool-augmented approaches (Gao et al., 2023; Chen et al., 2023) offload computation to code interpreters. More recently, reinforcement learning from outcome feedback has produced strong math-specialist models (DeepSeek-AI, 2025; He et al., 2025), and math-focused pretraining (Shao et al., 2024; Yang et al., 2024) advances competition benchmarks like MATH (Hendrycks et al., 2021). Unlike these methods, CFR restructures the *information available to the solver* rather than modifying the reasoning algorithm or training procedure.

O.2 Constraint-Based and Neuro-Symbolic Reasoning

Classical constraint satisfaction (Dechter, 2003) prunes infeasible regions before search, and neuro-symbolic approaches integrate symbolic solvers with neural models. In the NLP context, lexically constrained decoding (Hokamp and Liu, 2017; Post and Vilar, 2018) enforces hard token-level constraints during generation via modified beam search. NeuroLogic decoding (Lu et al., 2021) extends this to logical conjunctions and disjunctions of constraints. CRANE (Banerjee et al., 2025) applies constrained generation specifically to reasoning tasks. Grammar-constrained decoding (Willard and Louf, 2023) ensures outputs conform to formal grammars. Unlike these approaches, CFR keeps constraints in natural-language prompt context and does not use a symbolic solver or constrained decoder. It is training-free and can be applied to black-box backbones, but its effectiveness depends on the backbone’s Stage 1 extraction quality.

O.3 Structured Prompting for Mathematics

Beyond the approaches discussed in the main text, several structured prompting strategies relate to CFR:

Decomposition approaches. Zhou et al. (2023) decompose complex problems into simpler sub-questions. While CFR also performs a preliminary

analysis step, the output is not a problem decomposition but a constraint specification—a fundamentally different structure that complements rather than replaces decomposition.

Program-aided reasoning. PAL (Gao et al., 2023) and PoT (Chen et al., 2023) translate problems into executable code, gaining correctness guarantees for computation. CFR provides a softer form of verification: constraints are not formally verified by a theorem prover but checked by the model during reasoning. This makes CFR applicable to problems that resist formalization (e.g., those requiring geometric insight or combinatorial arguments).

Process supervision. Lightman et al. (2024) train process reward models (PRMs) to evaluate each reasoning step. CFR’s interleaved constraint checking can be seen as a prompt-based approximation of process supervision: the constraint specification acts as a lightweight “reward model” that the solver uses to self-evaluate intermediate results.

Structured representations beyond text. Structured object representations for vision-language spatial reasoning (Ma et al., 2026) and plan-generate-verify loops for parametric CAD editing (Ma et al.) similarly preserve task-relevant structure across multi-step generation. These are distinct modalities and tasks, but they motivate studying whether an explicit intermediate specification can improve reliability beyond mathematical text.

O.4 Constrained Reasoning in Classical AI

Constraint satisfaction. The CSP framework (Dechter, 2003) is a useful analogy, but not a theoretical foundation for CFR. Stage 1 extracts and summarizes restrictions in natural language, and Stage 2 checks them in a prompt; neither step implements arc consistency or formal backtracking.

Planning with constraints. Classical AI planners (STRIPS, PDDL) maintain preconditions and postconditions for actions. CFR’s interleaved checking is analogous to precondition verification: before proceeding to the next reasoning step, the solver confirms that the current state satisfies all known constraints.

O.5 Test-Time Computation

Scaling laws. Snell et al. (2025) show that allocating more test-time compute can be more effective than scaling model parameters. CFR is a targeted allocation of additional compute, but Table 3 shows that its gains must be evaluated jointly with its non-trivial token overhead; we make no equivalence claim to scaling model size.

Budget allocation. Muennighoff et al. (2025) explore simple test-time scaling through longer thinking. CFR provides a structured way to allocate the additional budget: rather than simply allowing the model to “think more,” it directs the extra computation toward constraint extraction, which has higher information density than additional free-form reasoning.

O.6 Inference-Time Interventions

Prompting as intervention. Our work contributes to the growing literature on inference-time interventions that improve reasoning without training. Unlike approaches that modify decoding (beam search variants, best-of- n) or post-process outputs (self-consistency, majority voting), CFR modifies the *information available to the model*. This places it closer to retrieval-augmented generation (RAG) conceptually—but instead of retrieving external knowledge, CFR extracts latent knowledge from the problem itself.

P Detailed Positive Cases by Constraint Type

We group the positive cases by the dominant constraint type exposed by the problem statement and required by the successful CFR trace. Each category first lists the detailed CoT–CFR comparison tables, followed by natural-language analyses that restate the problem, answer, and the specific constraint mechanism.

P.1 Geometric Configuration and Branch Constraints

These cases test whether the model can preserve orientation, cyclic order, sign choices, vector closure, and metric branch conditions before doing algebra. CFR helps by making geometric feasibility constraints explicit, especially when several algebraic branches look locally plausible.

P.1.1 Detailed Tables

Table 29: Detailed positive case 1 where CFR corrects CoT by enforcing explicit constraints.

Example 1: Ratio-and-Reflection Geometry (DeepSeek-32K, AIME 2025)		31.1% token reduction
Problem. In $\triangle ABC$, points D, E lie on $\overline{AB}$ and points F, G lie on $\overline{AC}$. Given $AD = 4, DE = 16, EB = 8, AF = 13, FG = 52, GC = 26$, let M be the reflection of D through F and N be the reflection of G through E . If $[DEGF] = 288$, find $[AFNBCEM]$.		
Ground Truth: 588 CoT: 33,065 tokens CFR: 22,791 tokens		
CoT	CFR	
CoT observes that D, F and E, G divide the two sides in the same ratios, but keeps the solution as a free-form geometric narrative. It does not turn the given area $[DEGF] = 288$ into the key constraint on $\sin A$, and the reflected points M, N are not explicitly encoded. As a result, the target heptagon is no longer controlled by the original construction, and the area computation drifts.	CFR first extracts the proportional structure: $\frac{AD}{AB} = \frac{AF}{AC} = \frac{1}{7}, \quad \frac{AE}{AB} = \frac{AG}{AC} = \frac{5}{7}.$ It then converts the area condition into a hard trigonometric constraint: $[DEGF] = 624 \sin A = 288 \quad \Rightarrow \quad \sin A = \frac{6}{13}.$ Finally, CFR encodes the reflections directly: $M = 2F - D, \quad N = 2E - G.$ Using shoelace on $AFNBCEM$ gives $[AFNBCEM] = 1274 \sin A = 588$.	
Answer: 1 $\times$	Answer: 588 $\checkmark$	

Table 30: Detailed positive case 2 where CFR corrects CoT by enforcing explicit constraints.

Example 2: Cyclic-Constraint Geometry (Qwen3.5-9B-32K, AIME 2024)		35.3% token reduction
Problem. Rectangles $ABCD$ and $EFGH$ are drawn such that D, E, C, F are collinear. Also, A, D, H, G all lie on a circle. If $BC = 16$, $AB = 107$, $FG = 17$, and $EF = 184$, find the length of CE .		
Ground Truth: 104 CoT: 33,015 tokens CFR: 21,370 tokens		
CoT	CFR	
CoT sets up coordinates but does not enforce the relative vertical orientation of the two rectangles. It treats the second rectangle as if it can be placed on the same side of the baseline without checking whether the concyclicity condition remains valid. As a result, the circle constraint is applied to the wrong geometric configuration, leading to an incorrect value of CE .	CFR first fixes the coordinate constraints: $D = (0, 0), \quad C = (107, 0), \quad A = (0, 16).$ It then uses the concyclicity of A, D, H, G to determine the correct orientation: H and G must lie on the opposite side of the baseline from A . $E = (x, 0), \quad F = (x + 184, 0),$ $H = (x, -17), \quad G = (x + 184, -17).$ The circle through A and D has center on $y = 8$, while chord HG forces the same center to have x -coordinate $x + 92$. $(x + 92)^2 + 8^2 = 92^2 + 25^2.$ Thus $(x + 92)^2 = 95^2$, so $x = 3$ and $CE = 107 - 3 = 104$.	
Answer: 64 $\times$	Answer: 104 $\checkmark$	

P.1.2 Natural-Language Analyses

Case 1: Ratio-and-reflection geometry. Problem. In a triangle, points on two sides are placed at specified segment lengths, the quadrilateral $DEGF$ has area 288, and points M, N are defined by reflections. The target is the area of $AFNBCEM$. **Answer.** 588. **Analysis.** The key is that the side ratios force parallel proportional structure, while the given area fixes $\sin A$. CFR succeeds because it writes these constraints and the reflection equations before computing the final area, preventing the solution from drifting away from the constructed polygon.

Case 2: Cyclic-constraint geometry. Problem. Two rectangles are arranged on the same baseline, and four vertices A, D, H, G are concyclic. Given the side lengths, find CE . **Answer.** 104. **Analysis.** The cyclic condition is not just a generic circle fact; it determines the relative vertical orientation of the two rectangles. CFR enforces this orientation first and then solves the circle-center equation, while CoT can easily apply concyclicity to the wrong configuration.

P.2 Algebraic, Modular, and Divisibility Constraints

These cases are governed by invariants, modular residues, divisibility filters, or extremal algebraic reductions. CFR is useful because it turns broad computation into a smaller feasible set determined by exact arithmetic constraints.

P.2.1 Detailed Tables

Table 31: Detailed positive case 3 where CFR corrects CoT by enforcing explicit constraints.

Example 3: Degree-Weighted Grid Optimization (DeepSeek-32K, AIME 2026)		69.2% token reduction
Problem. The integers from 1 to 64 are placed into an 8×8 grid. Let M be the sum of absolute differences across all adjacent horizontal and vertical cell pairs. Find the maximum possible value of M modulo 1000.		
Ground Truth: 896 CoT: 33,069 tokens CFR: 10,192 tokens		
CoT	CFR	
CoT relies on vague checkerboard intuition and memory of a similar problem rather than formalizing the optimization. It does not convert the grid into a bipartite graph with vertex-degree weights, so the assignment of large and small numbers remains heuristic. The reasoning stalls near the token limit and does not produce a reliable modular maximum.	CFR models the grid as a bipartite graph. The two parts each contain 32 vertices, with degree counts 18 of degree 4, 12 of degree 3, 2 of degree 2. The maximum is achieved by placing the 32 largest numbers on one side and the 32 smallest on the other, ordered by degree. High contribution = $4(999) + 3(486) + 2(67) = 5588$, Low contribution = $4(171) + 3(294) + 2(63) = 1692$. Thus $M_{\max} = 5588 - 1692 = 3896 \equiv 896 \pmod{1000}$.	
Answer: invalid $\times$	Answer: 896 $\checkmark$	

Table 32: Detailed positive case 4 where CFR corrects CoT by enforcing explicit constraints.

Example 4: Rational Recurrence Invariant (DeepSeek-32K, AIME 2025)		60.6% token reduction
Problem. Let $x_1 = \frac{25}{11}$ and $x_{k+1} = \frac{1}{3} \left(x_k + \frac{1}{x_k} - 1 \right)$. If $x_{2025} = \frac{m}{n}$ in lowest terms, find the remainder when $m + n$ is divided by 1000.		
Ground Truth: 248 CoT: 33,021 tokens CFR: 12,999 tokens		
CoT	CFR	
CoT expands the first few rational terms directly. The numerators and denominators grow quickly, and the derivation becomes trapped in arithmetic rather than structure.	CFR introduces $x_k = p_k/q_k$ in lowest terms and tracks	
It does not identify the invariant quantity that controls $m + n$, so the long computation leads to an incorrect small answer.	$S_k = p_k + q_k.$	
	The recurrence implies	
	$p_{k+1} = \frac{p_k^2 - p_k q_k + q_k^2}{3}, \quad q_{k+1} = p_k q_k,$	
	so	
	$S_{k+1} = \frac{(p_k + q_k)^2}{3} = \frac{S_k^2}{3}.$	
	Thus $S_{2025} = 2^{2^{2025}} 3^{2^{2024}+1}$. Reducing modulo 8 and 125, then applying CRT, gives 248.	
Answer: 2 ×	Answer: 248 ✓	

P.2.2 Natural-Language Analyses

Case 3: Degree-weighted grid optimization.

Problem. Three distinguished lattice points in a 7×7 grid generate a value by summing squared distances from all grid points; the task is to find the maximum value modulo 1000. **Answer.** 896.

Analysis. This problem is an optimization over a finite grid, so the structural constraint is the degree-weighted contribution of each chosen point. CFR converts the objective into a separable weighted-distance expression and selects the corner-extreme configuration, avoiding unsupported local search.

Case 4: Rational recurrence invariant. Problem.

A rational recurrence starts from $x_1 = 25/11$, and one must compute the remainder of $m+n$ when $x_{2025} = m/n$ is in lowest terms. **Answer.** 248.

Analysis. The recurrence looks computationally impossible if expanded directly. CFR identifies the invariant for $p_k + q_k$ under the recurrence, turning a long rational iteration into modular exponentiation.

P.3 Functional, Probabilistic, and Locus Constraints

These cases require splitting a function, probability distribution, or locus into valid branches. CFR improves reliability by forcing the model to check domains, endpoint exclusions, conditional cases, and real-valued feasibility before aggregating results.

P.3.1 Detailed Tables

Table 33: Detailed positive case 5 where CFR corrects CoT by enforcing explicit constraints.

Example 5: Sawtooth-Parabola Intersections (DeepSeek-32K, AIME 2025)		24.8% token reduction
Problem. A periodic sawtooth function f intersects the parabola $x = 34y^2$. The sum of all intersection y -coordinates is $\frac{a+b\sqrt{c}}{d}$. Find $a + b + c + d$.		
Ground Truth: 259 CoT: 33,700 tokens CFR: 25,346 tokens		
CoT	CFR	
CoT tries to solve the intersection equation across many periods, but the valid interval constraints are not consistently enforced. It over-accumulates candidate roots and produces a nonsensical large integer instead of the requested radical-form parameter sum.	CFR uses $x = 34y^2$ and $y = f(x)$, so $y \in [-1, 1]$ and $x \in [0, 34]$. It splits $x = 4m + t$ into the two sawtooth branches. On one branch, the quadratic has root sum $1/34$; on the other, the root sum is $-1/34$. These sums cancel across complete periods. Only the boundary period remains, where one root is invalid and must be discarded. The sum is $\frac{1 + 5\sqrt{185}}{68},$ so $a + b + c + d = 1 + 5 + 185 + 68 = 259$.	
Answer: 487372 $\times$	Answer: 259 $\checkmark$	

Table 34: Detailed positive case 6 where CFR corrects CoT by enforcing explicit constraints.

Example 6: Random Function Expectation (DeepSeek-32K, CMIMC 2025) +229.0% tokens	
Problem. For a uniformly random function $f : \{1, \dots, 25\} \rightarrow \{1, \dots, 25\}$, compute $\mathbb{E} \sum_{x=1}^{25} (f(f(x)) - x)^2$.	
Ground Truth: 2496 CoT: 5,376 tokens CFR: 17,687 tokens	
CoT	CFR
CoT computes the term for $x = 1$ and multiplies by 25, incorrectly assuming all summands have the same distribution. This ignores that the term $(f(x) - x)^2$ depends on the value of x, so the apparent symmetry is false.	CFR keeps x symbolic. For fixed x, let $Z = f(f(x))$. Then $Z = x \text{ with probability } \frac{1}{25},$ and otherwise Z is uniform on $\{1, \dots, 25\}$ with probability $\frac{24}{25}$. Thus $\mathbb{E}(Z - x)^2 = \frac{24}{25} (52 + (13 - x)^2).$ Summing over $x = 1, \dots, 25$ gives $\frac{24}{25} \sum_{x=1}^{25} (52 + (13 - x)^2) = 2496.$
Answer: 4704 $\times$	Answer: 2496 $\checkmark$

P.3.2 Natural-Language Analyses

Case 5: Sawtooth-parabola intersections.

Problem. A periodic sawtooth function f intersects the parabola $x = 34y^2$; the sum of all intersection y -coordinates is written as $\frac{a+b\sqrt{c}}{d}$, and the task asks for $a + b + c + d$. **Answer.** 259. **Analysis.** The problem becomes tractable only after splitting the sawtooth function into its valid branches and enforcing $y \in [-1, 1]$. CFR respects the interval constraints, cancels complete-period root sums, and keeps only the boundary contribution that determines the radical-form answer.

Case 6: Random function expectation. Problem.

For a uniformly random function $f : \{1, \dots, 25\} \rightarrow \{1, \dots, 25\}$, compute $\mathbb{E} \sum_x (f(f(x)) - x)^2$. **Answer.** 2496. **Analysis.** The important constraint is conditional: $f(f(x))$ equals x with one probability and otherwise behaves uniformly. CFR keeps x symbolic, so the variance-like term $(13 - x)^2$ is preserved instead of being washed out by a false symmetry argument.

P.4 Discrete Enumeration, Symmetry, and Graph Constraints

These cases are dominated by finite structure: block decompositions, symmetry classes, graph correspondences, maximality, and forced rows or columns. CFR helps by extracting the combinatorial object first, which reduces overcounting and prevents invalid configurations from entering the count.

P.4.1 Detailed Tables

Table 35: Detailed positive case 7 where CFR corrects CoT by enforcing explicit constraints.

Example 7: Semi-Magic Square Counting (DeepSeek-32K, BRUMO 2025)		67.2% token reduction
Problem. Digits 1 through 9 are placed in a 3×3 grid so that all rows and columns have the same sum. Diagonals need not have the same sum. Count the number of valid grids.		
Ground Truth: 72 CoT: 32,971 tokens CFR: 10,806 tokens		
CoT	CFR	
CoT correctly observes that the common row and column sum must be 15, but then drifts among known facts about magic squares, Latin squares, and semi-magic squares. The trace repeatedly second-guesses whether the count should be 72, 288, or a related known value, leaving the answer unstable.	CFR fixes the invariant first: since $1 + 2 + \dots + 9 = 45$, each row and column must sum to 15. It then enumerates the feasible triples: $\{1, 5, 9\}, \{1, 6, 8\}, \{2, 4, 9\}, \{2, 5, 8\},$ $\{2, 6, 7\}, \{3, 4, 8\}, \{3, 5, 7\}, \{4, 5, 6\}.$ A systematic partition-and-permutation count gives exactly 72 semi-magic grids.	
Answer: unstable $\times$	Answer: 72 $\checkmark$	

Table 36: Detailed positive case 8 where CFR corrects CoT by enforcing explicit constraints.

Example 8: Sudoku-Style Counting (Qwen3.5-9B-32K, AIME 2025)		+2.4% tokens
Problem. The 27 cells of a 3×9 Sudoku-style band are filled using digits 1 through 9 so that every row and each 3×3 block contains all digits. If the count is $p^a q^b r^c s^d$, compute $pa + qb + rc + sd$.		
Ground Truth: 81 CoT: 33,446 tokens CFR: 34,247 tokens		
CoT	CFR	
CoT fails to return a stable parsed answer and does not cleanly isolate the structural count of the middle block. The key missing component is the number of feasible second-block set assignments after the first block has fixed row-wise digit sets.	CFR decomposes the grid by 3×3 blocks. The first block has $9!$ fillings. Given the first block, the second block requires choosing row sets that avoid the corresponding first-block row sets and still partition $\{1, \dots, 9\}$. This contributes the structural multiplier 56. $N = 9! \cdot 56 \cdot (3!)^3 \cdot (3!)^3.$ Therefore $N = 2^{16} 3^{10} 5 \cdot 7^2$, and $2 \cdot 16 + 3 \cdot 10 + 5 + 7 \cdot 2 = 81.$	
Answer: None ×	Answer: 81 ✓	

P.4.2 Natural-Language Analyses

Case 7: Semi-magic square counting. Problem.

Place digits 1 through 9 in a 3×3 grid so that all rows and columns have the same sum, while diagonals are unrestricted; count the valid grids.

Answer. 72. **Analysis.** The total sum forces every row and column to sum to 15. CFR first fixes this invariant and enumerates the feasible triples before counting partitions and permutations, which avoids drifting between magic-square facts and the weaker semi-magic condition.

Case 8: Sudoku-style counting. Problem.

Count valid 3×9 Sudoku-style fillings and express the result as a prime factorization, then compute the requested weighted exponent sum. **Answer.** 81.

Analysis. The decisive missing factor is the number of feasible middle-block assignments. CFR keeps the block-compatibility constraint explicit, inserts the multiplier 56, and therefore factors the correct total rather than an incomplete count.

Q Full Constraint Taxonomy with Problem Examples

For each constraint type, we provide a canonical AIME problem example and the expected Stage 1 output.

Q.1 Domain Constraints

Example: “Find the number of positive integers $n < 1000$.”

Extracted:

C1: type=domain, desc="n is a positive integer"
C2: type=bound, desc="n < 1000"
C3: type=domain, desc="answer is a count (non-negative integer)"

Propagated:

P1: n in {1, 2, ..., 999}, answer in {0, 1, ..., 999}

Q.2 Modular Constraints

Example: “...find the remainder when N is divided by 1000.”

Extracted:

C1: type=modular, desc="final answer = N mod 1000"
C2: type=bound, desc="answer in [0, 999]"
C3: type=domain, desc="answer is an integer"

Propagated:

P1: answer is an integer in [0, 999]

Critical: Must apply mod 1000 as the very last step.

Q.3 Format Constraints

Example: “...can be expressed as $\frac{p}{q}$ where $\gcd(p, q) = 1$. Find $p + q$.”

Extracted:

C1: type=format, desc="express as p/q with $\gcd(p, q) = 1$ "
C2: type=format, desc="final answer = p + q"
C3: type=domain, desc="p, q are positive integers"

Propagated:

P1: answer = p+q where p/q is in lowest terms, answer > 1

Critical: Must reduce fraction before computing p+q.

Q.4 Parity Constraints

Example: “Find the number of even positive integers $n \leq 100$.”

Extracted:

C1: type=parity, desc="n must be even"
C2: type=domain, desc="n is a positive integer"
C3: type=bound, desc="n ≤ 100"

Propagated:

P1: n in {2, 4, 6, ..., 100}, 50 candidates

Critical: Only enumerate even values.

Q.5 Bound Constraints

Example: “Find the largest integer n such that $n^3 < 10000$.”

Extracted:

C1: type=bound, desc="n^3 < 10000"
C2: type=domain, desc="n is an integer"
C3: type=bounds_or_extremal, desc="find largest such n"

Propagated:

P1: n ≤ 21 (since $21^3 = 9261 < 10000$, $22^3 = 10648 > 10000$)

Critical: Answer is 21, must verify both directions.

Q.6 Structure Constraints

Example: “...the number of permutations of $\{1, \dots, n\}$ that are involutions.”

Extracted:

C1: type=structure, desc="permutation must be an involution ($p(p(i)) = i$)"
C2: type=domain, desc="answer is a count"
C3: type=structure, desc="involutions consist of fixed points and 2-cycles"

Propagated:

P1: answer = sum over k of $C(n, 2k) * (2k-1)!!$

Critical: Do not count general permutations.

R Broader Impact

CFR is a purely methodological contribution to mathematical reasoning. It does not involve data collection from human subjects or new privacy-sensitive data. By reducing some constraint-violation errors, it may improve the reliability of AI-assisted mathematics; however, its summaries can also be incomplete or invalid, so it should not be used as a guarantee of mathematical correctness. The technique is training-free and does not require parameter updates, but its effectiveness varies with backbone quality and adds inference cost.

Although CFR is not an interactive or safety-critical system, work on red-teaming policies and harm amplification in LLM interactions ([Guo et al., 2026a,b](#)) underscores the importance of evaluating any intervention within its stated scope and avoiding claims that extend beyond the measured setting.

A potential concern is that CFR could contribute to overreliance on AI for mathematical problem-solving in educational settings. We note that CFR’s explicit constraint-extraction step could also serve as a pedagogical tool: students could learn systematic constraint identification by examining CFR’s Stage 1 outputs.

S Future Directions

We outline several promising directions for extending CFR beyond the current work.

Learned constraint extraction. The current Stage 1 relies entirely on prompting. Fine-tuning a lightweight model specifically for constraint extraction could improve both precision and speed. A distilled 1B-parameter extractor, trained on $(problem, constraints)$ pairs generated by DeepSeek-V4-Pro, could serve as a universal plug-in for any downstream solver at minimal cost.

Multi-turn constraint refinement. Currently, constraints are extracted once and passed to the solver without further interaction. A multi-turn variant could allow the solver to query the constraint module when encountering ambiguity—e.g., “Does constraint C3 apply to intermediate variable k or only to the final answer?” This would address 23% of our observed failure cases where constraint scope was misinterpreted. Interactive-guidance policy optimization, heterogeneous-agent collaboration, and hierarchical negative feedback for personalized styling offer related directions for structuring iterative feedback (Liu et al., 2026; Zhang et al., 2026; Ma et al., 2025), but are outside the scope of the present training-free method.

Domain extension beyond mathematics. The core principle—extract structural constraints before solving—applies to any domain with well-defined answer formats. Related work uses LLM-driven semantic disambiguation and dynamic visual focus in medical diagnosis (Zhu et al., 2025, 2026); these results motivate, but do not establish, a corresponding extension of CFR beyond mathematical text:

- **Code generation:** Extract type signatures, input/output format constraints, and edge-case specifications before generating code.
- **Legal reasoning:** Extract jurisdictional constraints, applicable statutes, and precedent requirements before drafting arguments.
- **Scientific computing:** Extract physical units, conservation laws, and boundary conditions before deriving formulas.
- **Formal verification:** Extract pre/post-conditions and loop invariants before synthesizing proofs.

Adaptive routing with soft scores. The current binary router (use CFR or not) could be extended

to a soft scorer that modulates the constraint extraction budget based on estimated constraint density. Problems with many detected cues would receive larger Stage 1 budgets; problems with few cues would receive minimal extraction. This complements work on whether reasoning models can identify when to stop allocating internal reasoning (Huang et al., 2026). Preliminary analysis suggests this could reduce average token overhead by 18% while maintaining accuracy.

Integration with search-based methods. CFR is orthogonal to tree search approaches like Tree of Thoughts (Yao et al., 2023a). Constraints could serve as pruning criteria: any branch producing an intermediate result inconsistent with propagated constraints is immediately abandoned. This is analogous to constraint propagation in classical CSP solvers (Dechter, 2003) and could dramatically reduce the effective branching factor.

Benchmark development. Our work highlights the need for benchmarks specifically designed to test constraint handling in LLMs. An ideal benchmark would include:

1. Problems with explicit constraints at varying complexity levels.
2. Problems with implicit constraints requiring inference.
3. Adversarial problems where surface-level constraint cues are misleading.
4. Multi-constraint problems where individual constraint satisfaction is easy but joint satisfaction requires careful reasoning.

Such a benchmark would enable more targeted evaluation of methods like CFR and drive further research in constraint-aware reasoning.

Theoretical analysis. A formal analysis of when constraint-first decomposition provably helps versus standard monolithic reasoning remains open. Key questions include: (1) Under what problem distributions does the constraint-guided solver have strictly lower error probability? (2) What is the optimal allocation of compute between extraction and solving as a function of constraint complexity? (3) Can we derive PAC-style bounds on the extraction quality needed to guarantee net positive gain?